

## Basic Questions (1-100) - Integer Data Type in Go

### 1-20: Basics of Integers

1. What is an `int` in Go, and how does it differ from `int8`, `int16`, `int32`, and `int64`?
2. What happens if you assign a floating-point value to an `int` variable?
3. How do you convert an `int` to a `string` in Go?
4. What will happen if you exceed the maximum value of an `int`?
5. How does Go handle integer overflow?
6. What is the default value of an `int` variable in Go?
7. What is the size of an `int` on a 32-bit system?
8. What is the size of an `int` on a 64-bit system?
9. What is the difference between `int` and `uint`?
10. Can an `int` store negative numbers?
11. What happens if you perform division between two integers in Go?
12. How do you round a floating-point number to the nearest integer in Go?
13. How do you check if an integer is even or odd in Go?
14. What is the `%` (modulus) operator used for in integer arithmetic?
15. How do you compare two integers in Go?
16. Can you assign an `int8` value to an `int` variable? Why or why not?
17. How do you generate a random integer between 1 and 100 in Go?
18. What is the result of `5 / 2` in Go? Why?
19. What happens if you try to divide an integer by zero in Go?
20. How do you convert a string to an integer in Go?

### 21-40: Integer Operations and Bitwise Operators

21. How do you add two integers in Go?
22. How do you subtract two integers in Go?
23. How do you multiply two integers in Go?
24. How do you find the absolute value of an integer in Go?
25. How do you swap two integer variables without using a temporary variable?
26. What is the result of `5 % 2` in Go?
27. How does integer division work in Go?
28. How do you increment and decrement an integer in Go?
29. What are bitwise operators in Go?
30. How do you perform a bitwise AND operation between two integers?
31. How do you perform a bitwise OR operation between two integers?
32. How do you perform a bitwise XOR operation between two integers?
33. How do you shift an integer 2 bits to the left in Go?
34. How do you shift an integer 3 bits to the right in Go?
35. What is a bit mask, and how is it used in integer operations?
36. How do you check if the `n`th bit of an integer is set?
37. How do you count the number of 1s in the binary representation of an integer?
38. How do you check if an integer is a power of two?
39. How do you find the highest power of two less than or equal to a given integer?

40.What happens if you left shift an integer beyond its bit size?

#### **41-60: Integer Limits and Special Cases**

- 41.What is the maximum value of an `int8`?
  - 42.What is the minimum value of an `int8`?
  - 43.What is the maximum value of an `int16`?
  - 44.What is the minimum value of an `int16`?
  - 45.What is the maximum value of an `int32`?
  - 46.What is the minimum value of an `int32`?
  - 47.What is the maximum value of an `int64`?
  - 48.What is the minimum value of an `int64`?
  - 49.How do you safely add two large integers without overflow?
  - 50.How do you safely subtract two large integers without underflow?
  - 51.How do you handle integer overflow in Go?
  - 52.How do you handle integer underflow in Go?
  - 53.What is `math.MaxInt` in Go?
  - 54.What is `math.MinInt` in Go?
  - 55.How do you check for integer overflow before performing an operation?
  - 56.How do you calculate the sum of digits of an integer?
  - 57.How do you reverse the digits of an integer?
  - 58.How do you determine if an integer is a palindrome?
  - 59.How do you find the greatest common divisor (GCD) of two integers?
  - 60.How do you find the least common multiple (LCM) of two integers?
- 

### **Advanced (101-150) - Integer Data Type in Go**

#### **101-120: Complex Mathematical Operations**

- 101.Implement a function to calculate the nth Fibonacci number using matrix exponentiation.
- 102.Implement a function to calculate factorial modulo a prime number.
- 103.Compute the nth prime number using an optimized prime sieve.
- 104.Implement a function to count trailing zeros in factorial.
- 105.Find the sum of divisors of n using a prime sieve for optimization.
- 106.Implement a function to check if an integer is a palindrome without converting to a string.
- 107.Compute the number of ways to express n as the sum of two primes.
- 108.Implement the Miller-Rabin primality test for large numbers.
- 109.Compute binomial coefficients efficiently using Pascal's Triangle.
- 110.Implement integer modular exponentiation using fast exponentiation.
- 111.Solve an integer linear programming problem using branch-and-bound.
- 112.Implement a function to compute the totient function efficiently for large n.
- 113.Implement a function to find the nth Stern's Diatomic Series number.
- 114.Compute the largest power of 2 that divides n without loops.
- 115.Implement a function that finds the smallest number with exactly k divisors.
- 116.Implement a function to generate integer partitions of n.

117. Compute Carmichael numbers up to  $n$ .
118. Implement integer-based radix sort for sorting large numbers.
119. Compute the integer square root of a large number using Newton's method.
120. Find the largest prime factor of  $n$  efficiently.

## Basic Questions (1-100) - String Data Type in Go

### 1-20: String Basics

1. What is a `string` in Go?
2. How do you declare a string variable in Go?
3. What is the default value of a string in Go?
4. How do you concatenate two strings in Go?
5. Can you modify a string directly in Go? Why or why not?
6. How do you find the length of a string in Go?
7. How do you access individual characters in a string?
8. What happens if you try to change a character in a string?
9. How do you convert a string to a byte slice in Go?
10. How do you convert a byte slice back to a string?
11. How do you check if a string is empty in Go?
12. How do you compare two strings in Go?
13. What happens if you compare two strings of different lengths?
14. How do you iterate over a string character by character?
15. How do you check if a string contains a specific substring?
16. How do you find the index of a substring within a string?
17. How do you replace all occurrences of a substring in a string?
18. How do you convert a string to uppercase in Go?
19. How do you convert a string to lowercase in Go?
20. How do you trim spaces from a string in Go?

### 21-40: String Operations

21. How do you check if a string starts with a specific prefix?
22. How do you check if a string ends with a specific suffix?
23. How do you split a string into a slice of substrings?
24. How do you join a slice of strings into a single string?
25. What is the difference between `strings.Split()` and `strings.Fields()`?
26. How do you count the number of occurrences of a character in a string?
27. How do you reverse a string in Go?
28. How do you check if a string is a palindrome?
29. How do you remove all whitespace characters from a string?
30. How do you remove a specific character from a string?
31. How do you replace only the first occurrence of a substring in a string?
32. How do you check if two strings are anagrams?
33. How do you find the longest common prefix of two strings?

- 34.How do you count the number of words in a string?
- 35.How do you extract a substring from a string in Go?
- 36.What is the `strings.Builder` type used for?
- 37.How do you optimize string concatenation in a loop?
- 38.How do you encode a string in Base64 in Go?
- 39.How do you decode a Base64 encoded string in Go?
- 40.How do you compare strings in a case-insensitive manner?

#### **41-60: Unicode and Special Cases**

- 41.What is UTF-8, and how does Go handle Unicode?
  - 42.How do you check if a string contains only ASCII characters?
  - 43.How do you check if a string contains non-ASCII characters?
  - 44.How do you remove non-ASCII characters from a string?
  - 45.How do you find the number of runes (Unicode characters) in a string?
  - 46.How do you extract runes from a string?
  - 47.What is the difference between `rune` and `byte` in Go?
  - 48.How do you convert a `rune` slice back into a string?
  - 49.How do you check if a string contains only numeric characters?
  - 50.How do you check if a string contains only alphabetic characters?
  - 51.How do you pad a string with spaces or zeros?
  - 52.How do you escape special characters in a string?
  - 53.How do you unescape special characters in a string?
  - 54.How do you remove diacritical marks (accents) from characters in a string?
  - 55.How do you normalize Unicode text in Go?
  - 56.How do you validate if a string is a well-formed email address?
  - 57.How do you validate if a string is a valid URL?
  - 58.How do you find all occurrences of a substring in a string?
  - 59.How do you extract numbers from a string?
  - 60.How do you remove duplicate words from a string?
- 

### **Advanced (101-150) - String Data Type in Go**

#### **101-120: Complex String Manipulation**

- 101.How do you implement a custom string tokenizer in Go?
- 102.How do you efficiently find the longest repeated substring in a string?
- 103.Implement a function to generate all permutations of a string.
- 104.How do you implement the Knuth-Morris-Pratt (KMP) string search algorithm in Go?
- 105.How do you implement the Rabin-Karp substring search algorithm?
- 106.How do you implement the Boyer-Moore string search algorithm?
- 107.Implement a function to find the longest palindromic substring in a string.
- 108.Implement a function to convert a string to Morse code and back.
- 109.Implement a function to find all anagram groups in a list of words.
- 110.Implement a function to perform text compression using Huffman coding.
- 111.How do you implement a function to remove all duplicate characters from a string?

112. Implement a function to check if one string is a rotation of another.
113. How do you efficiently remove stop words from a paragraph?
114. Implement a function to find the most frequently occurring word in a paragraph.
115. Implement a function to sort words in a string by length.
116. How do you implement a function to count the number of vowels and consonants in a string?
117. Implement a function to detect and correct spelling errors in a string.
118. Implement a function to auto-capitalise the first letter of every sentence in a paragraph.
119. Implement a function to encode a string using Run-Length Encoding (RLE).
120. Implement a function to perform URL encoding and decoding.

## Basic Questions (1-100) - Float Data Type in Go

### 1-20: Basics of Floating-Point Numbers

1. What is a `float` in Go, and how does it differ from `float32` and `float64`?
2. What is the default value of a `float` variable in Go?
3. What is the difference between `float32` and `float64`?
4. How do you declare a floating-point variable in Go?
5. How do you assign a floating-point number to a variable?
6. How do you perform addition, subtraction, multiplication, and division on floating-point numbers?
7. Can you store an integer in a `float` variable? Why or why not?
8. How do you convert an integer to a floating-point number?
9. How do you convert a floating-point number to an integer?
10. What happens when you divide a number by zero in Go?
11. What is the value of `math.Pi` in Go?
12. What happens when you assign a very large floating-point number to a `float32` variable?
13. How do you check if a floating-point number is equal to another?
14. Why is comparing floating-point numbers directly often problematic?
15. What is NaN (Not-a-Number) in Go, and how do you check for it?
16. What is positive and negative infinity in Go?
17. How do you generate a random floating-point number in Go?
18. How do you find the square root of a floating-point number in Go?
19. How do you calculate the power of a number in Go?
20. How do you find the absolute value of a floating-point number?

### 21-40: Floating-Point Precision and Rounding

21. How do you round a floating-point number to the nearest integer?
22. How do you round a floating-point number to two decimal places?
23. What is the smallest positive nonzero `float64` value in Go?
24. What is the largest `float64` value that Go can represent?
25. How do you check if a floating-point number is finite?

26. How do you handle floating-point arithmetic errors in Go?
27. What is floating-point precision loss?
28. Why do some floating-point numbers not have an exact representation in binary?
29. How do you ensure precision when working with financial calculations in Go?
30. How do you compare two floating-point numbers within a tolerance?
31. How do you truncate a floating-point number without rounding?
32. How do you format a floating-point number as a string with a fixed number of decimal places?
33. How do you convert a floating-point number to a string in Go?
34. How do you convert a string to a floating-point number in Go?
35. What is `math.Trunc()` used for in Go?
36. How do you implement a function to check if a floating-point number is an integer?
37. How do you determine if a floating-point number is a whole number?
38. How do you calculate the ceiling of a floating-point number?
39. How do you calculate the floor of a floating-point number?
40. How do you extract the fractional part of a floating-point number?

#### **41-60: Advanced Floating-Point Operations**

41. How do you compute the logarithm of a floating-point number in Go?
42. What is the difference between `math.Log()` and `math.Log10()`?
43. How do you compute the exponential of a floating-point number?
44. How do you compute the sine of an angle in radians?
45. How do you compute the cosine of an angle in radians?
46. How do you compute the tangent of an angle in radians?
47. How do you convert degrees to radians in Go?
48. How do you convert radians to degrees in Go?
49. How do you calculate the hypotenuse of a right triangle using floating-point numbers?
50. How do you generate a random floating-point number between two given numbers?
51. How do you check if a floating-point number is within a specific range?
52. How do you add two floating-point numbers safely without overflow?
53. How do you subtract two floating-point numbers safely without underflow?
54. How do you divide two floating-point numbers safely?
55. How do you detect floating-point overflow and underflow in Go?
56. How do you implement an efficient floating-point-based sorting function?
57. How do you find the maximum floating-point value in a slice?
58. How do you find the minimum floating-point value in a slice?
59. How do you compute the arithmetic mean of a slice of floating-point numbers?
60. How do you compute the standard deviation of a slice of floating-point numbers?

---

## **Advanced (101-150) - Float Data Type in Go**

### **101-120: Complex Mathematical Computations**

101. Implement a function to approximate the value of Pi using a Monte Carlo method.
102. Implement a function to solve quadratic equations using floating-point arithmetic.

103. Implement an efficient algorithm for computing floating-point matrix multiplication.
104. Implement a function to compute the natural logarithm using the Taylor series.
105. Implement a function to compute the sine and cosine using the Taylor series.
106. Implement a floating-point number parser that handles special cases like NaN and Infinity.
107. Compute the moving average of a large stream of floating-point numbers.
108. Implement a function to compute the Nth root of a number using Newton's method.
109. Implement a function to compute the power of a floating-point number using fast exponentiation.
110. Implement a function to approximate e (Euler's number) using a series expansion.
111. Compute the floating-point determinant of an NxN matrix.
112. Implement a function to convert a floating-point number into a fraction.
113. Implement a function to find the best rational approximation of a floating-point number.
114. Implement a function to simulate floating-point errors in real-world computations.
115. Implement a function to check if a floating-point number can be represented exactly in binary.
116. Implement a function to compute the geometric mean of a set of numbers.
117. Implement an adaptive floating-point precision function for scientific computations.
118. Implement a function to efficiently generate a sequence of prime numbers using floating-point calculations.
119. Implement a function to determine if a floating-point number is nearly an integer.
120. Implement an optimized method to compute the area of a polygon using floating-point numbers.

## Basic Questions (1-100) - Map Data Type in Go

### 1-20: Basics of Maps

1. What is a map in Go, and how does it differ from an array or slice?
2. How do you declare a map in Go?
3. How do you initialize an empty map?
4. How do you create a map with predefined key-value pairs?
5. What data types can be used as keys in a Go map?
6. Can a slice be used as a key in a Go map? Why or why not?
7. How do you add a key-value pair to a map?
8. How do you retrieve a value from a map?
9. What happens when you try to access a key that does not exist in a map?
10. How do you check if a key exists in a map?
11. How do you update the value of a key in a map?
12. How do you delete a key-value pair from a map?
13. What is the zero value of a map in Go?
14. Can a map contain duplicate keys? Why or why not?
15. How do you find the length (number of key-value pairs) of a map?
16. How do you iterate over a map using a for loop?
17. Does the order of elements in a map remain the same? Why or why not?

- 18.How do you clear all elements in a map?
- 19.How do you check if a map is empty?
- 20.Can a Go map contain `nil` values?

## **21-40: Advanced Map Operations**

- 21.How do you merge two maps in Go?
- 22.How do you copy a map in Go?
- 23.What happens when you assign one map variable to another?
- 24.How do you compare two maps for equality?
- 25.Can a map store another map as a value? How?
- 26.How do you sort the keys of a map?
- 27.How do you sort the values of a map?
- 28.How do you create a map where the key is a struct?
- 29.How do you create a map where the value is a struct?
- 30.How do you convert a map to a JSON string?
- 31.How do you convert a JSON string to a map?
- 32.How do you handle concurrent access to a map in Go?
- 33.What is `sync.Map`, and how does it differ from a regular map?
- 34.How do you use `sync.Map` for concurrent operations?
- 35.How do you find the key with the highest value in a map?
- 36.How do you find the key with the lowest value in a map?
- 37.How do you remove multiple keys from a map?
- 38.How do you count occurrences of elements using a map?
- 39.How do you use a map as a cache?
- 40.What are the memory considerations when working with large maps?

## **41-60: Working with Nested Maps and Performance Optimization**

- 41.How do you create a map where the value is a slice?
- 42.How do you add values to a map where the value is a slice?
- 43.How do you remove a specific value from a map where the value is a slice?
- 44.How do you create a map with default values for missing keys?
- 45.How do you handle `nil` maps safely in Go?
- 46.What is the performance complexity of searching for a key in a map?
- 47.What is the performance complexity of inserting a key-value pair in a map?
- 48.What is the performance complexity of deleting a key from a map?
- 49.How does Go internally manage hash collisions in maps?
- 50.How do you choose between a `map` and a `slice` for storing data?
- 51.How do you choose between a `map` and a `struct` for storing data?
- 52.How do you implement a Least Recently Used (LRU) cache using a map?
- 53.How do you count unique elements in a slice using a map?
- 54.How do you implement a frequency counter using a map?
- 55.How do you safely read from a map in a multi-threaded application?
- 56.How do you safely write to a map in a multi-threaded application?
- 57.How does garbage collection affect maps in Go?
- 58.What are some common memory leaks related to maps in Go?



- 59.How do you manually free memory used by a large map?
  - 60.How does Go handle resizing of maps internally?
- 

## Advanced (101-150) - Map Data Type in Go

### 101-120: Complex Map Manipulations

- 101.Implement a function to reverse the key-value pairs of a map.
- 102.Implement a function to find all keys associated with a given value in a map.
- 103.Implement a function to get the top N highest values in a map.
- 104.Implement a function to remove duplicate values from a map.
- 105.Implement a function to group elements of a slice by a specific criterion using a map.
- 106.Implement a function to flatten a nested map into a single-level map.
- 107.Implement a function to calculate the histogram of characters in a string using a map.
- 108.Implement a function to generate a map of Fibonacci numbers up to N.
- 109.Implement a function to perform set operations (union, intersection, difference) using maps.
- 110.Implement a function to find the most common word in a large text using a map.
- 111.Implement a function to convert a map into a custom struct.
- 112.Implement a function to check if two maps are anagrams of each other.
- 113.Implement a function to partition a map based on a condition.
- 114.Implement a function to compress a map by combining values with similar keys.
- 115.Implement a function to efficiently store and retrieve key-value pairs from a large dataset.
- 116.Implement a function to calculate the Levenshtein distance between two words using a map.
- 117.Implement a function to simulate a trie (prefix tree) using nested maps.
- 118.Implement a function to perform fast auto-suggestions using a map.
- 119.Implement a function to serialize and deserialize a deeply nested map structure.
- 120.Implement a function to efficiently find all anagrams in a list using a map.

## Basic Questions (1-100) - Boolean Data Type in Go

### 1-20: Boolean Basics

- 1. What is a `bool` type in Go?
- 2. What are the possible values of a `bool` variable in Go?
- 3. What is the default value of a `bool` variable in Go?
- 4. How do you declare a boolean variable in Go?
- 5. How do you initialize a boolean variable with `true` or `false`?
- 6. How do you use the `bool` type in an `if` statement?
- 7. Can a boolean be converted to an integer in Go? Why or why not?
- 8. Can an integer be converted to a boolean in Go?
- 9. How do you negate a boolean value in Go?

10. What is the result of `!true` in Go?
11. What is the result of `!false` in Go?
12. How do you compare two boolean values?
13. How do you use boolean variables with logical operators?
14. What does the `&&` (AND) operator do with boolean values?
15. What does the `||` (OR) operator do with boolean values?
16. What is the result of `true && false` in Go?
17. What is the result of `true || false` in Go?
18. How does short-circuit evaluation work with `&&` and `||` in Go?
19. How do you use boolean values in a `switch` statement?
20. Can a boolean be used in an arithmetic operation in Go?

## **21-40: Boolean Logic and Expressions**

21. What is De Morgan's Law in boolean algebra?
22. How do you simplify `!(A && B)` using De Morgan's Law?
23. How do you simplify `!(A || B)` using De Morgan's Law?
24. How do you check if two conditions are both `true`?
25. How do you check if at least one condition is `true`?
26. How do you determine whether a number is even using a boolean expression?
27. How do you determine whether a number is odd using a boolean expression?
28. What is the difference between `==` and `!=` with boolean values?
29. What is the result of `true == false` in Go?
30. What is the result of `true != false` in Go?
31. How do you combine multiple boolean conditions?
32. How do you write a function that returns a boolean value?
33. How do you use a boolean return value in an `if` statement?
34. How do you write a function that checks if a string is empty and returns a boolean?
35. How do you check if a slice is empty using a boolean expression?
36. How do you check if a map is empty using a boolean expression?
37. How do you check if an array contains a specific element using a boolean expression?
38. How do you use boolean values in loops?
39. How do you terminate a loop based on a boolean condition?
40. How do you use boolean flags in Go?

## **41-60: Advanced Boolean Operations**

41. How do you compare two boolean expressions in Go?
42. How do you use boolean values in a struct?
43. How do you use a boolean field in a struct for feature toggling?
44. How do you toggle a boolean value in Go?
45. How do you convert a boolean to a string in Go?
46. How do you convert a string to a boolean in Go?
47. How do you use boolean values in JSON encoding?
48. How do you use boolean values in JSON decoding?
49. How does `reflect.TypeOf()` behave with boolean variables?

50. How do you implement a boolean toggle function in Go?
  51. How do you count the number of `true` values in a boolean slice?
  52. How do you check if all values in a boolean slice are `true`?
  53. How do you check if at least one value in a boolean slice is `true`?
  54. How do you use boolean conditions to filter elements from a slice?
  55. How do you use boolean flags in a command-line application?
  56. How do you implement a retry mechanism using boolean values?
  57. How do you optimize boolean expressions for performance?
  58. How do you benchmark boolean expressions in Go?
  59. What are the common pitfalls when working with boolean values?
  60. What is the zero value of a `bool` array in Go?
- 

## Advanced (101-150) - Boolean Data Type in Go

### 101-120: Complex Boolean Logic and Use Cases

101. Implement a function that evaluates a complex boolean expression.
102. Implement a function that performs XOR using only `&&`, `|`, and `!`.
103. Implement a function that finds the majority boolean value in a slice.
104. Implement a function that checks if two boolean slices are equal.
105. Implement a function that merges two boolean slices using AND logic.
106. Implement a function that merges two boolean slices using OR logic.
107. Implement a function that finds the longest sequence of `true` values in a boolean slice.
108. Implement a function that simulates a light switch using a boolean variable.
109. Implement a function that simulates a security system using boolean logic.
110. Implement a function that simulates a voting system using boolean logic.
111. Implement a function that filters a list based on multiple boolean conditions.
112. Implement a function that dynamically builds boolean expressions.
113. Implement a function that performs a binary search based on a boolean condition.
114. Implement a function that calculates boolean probability from a dataset.
115. Implement a function that performs pattern matching using boolean values.
116. Implement a function that finds contradictions in a boolean dataset.
117. Implement a function that builds a truth table for a given boolean expression.
118. Implement a function that simplifies boolean expressions using logical identities.
119. Implement a function that converts a boolean slice to a bitmask representation.
120. Implement a function that converts a bitmask representation back to a boolean slice.

## Basic Questions (1-100) - Pointer Data Type in Go

### 1-20: Pointer Basics

1. What is a pointer in Go?
2. How do you declare a pointer variable in Go?

3. How do you initialize a pointer with a memory address?
4. What does the `*` operator do when used with a pointer?
5. What does the `&` operator do in Go?
6. What is the zero value of a pointer in Go?
7. How do you check if a pointer is `nil`?
8. What happens when you dereference a `nil` pointer?
9. How do you assign one pointer variable to another?
10. Can a pointer store the address of another pointer?
11. How do you pass a pointer to a function?
12. How do you modify a value inside a function using pointers?
13. What is the difference between passing a variable by value and by pointer?
14. Can a pointer be used as a return type of a function?
15. What is a pointer to a pointer in Go?
16. How do you dynamically allocate memory using a pointer in Go?
17. How do you compare two pointers?
18. Can you perform arithmetic operations on pointers in Go?
19. How do you create a slice of pointers in Go?
20. Can you store pointers inside a map in Go?

#### **21-40: Advanced Pointer Usage**

21. How do you initialize a pointer using `new()`?
22. How does `new()` differ from using `&` to create a pointer?
23. What is the difference between `new()` and `make()` in Go?
24. How do you return a pointer to a local variable from a function?
25. Why does Go allow returning pointers to local variables?
26. How do you update a struct field using a pointer?
27. How do you create a method that modifies a struct using a pointer receiver?
28. What is a pointer receiver in Go?
29. What is the advantage of using a pointer receiver over a value receiver?
30. Can you convert a pointer type to another type in Go?
31. Can you create an array of pointers?
32. How do you loop through an array of pointers?
33. How do you loop through a slice of pointers?
34. How do you swap two values using pointers?
35. How do you implement a function that resets a variable to its zero value using pointers?
36. Can you use pointers with interfaces in Go?
37. What happens when you assign a `nil` pointer to an interface?
38. How do you safely use pointers in multi-threaded programs?
39. Can a pointer reference an interface type?
40. How does Go handle garbage collection of pointer-allocated memory?

#### **41-60: Pointers and Structs**

41. How do you create a pointer to a struct?
42. How do you initialize a struct using a pointer?
43. What is the difference between `var p *Person` and `p := &Person{}`?

- 44.How do you access struct fields using a pointer?
  - 45.How do you modify struct fields through a pointer?
  - 46.How do you copy a struct while preserving a pointer field?
  - 47.Can you store a pointer to a struct inside another struct?
  - 48.What happens when you assign a struct with pointer fields to another struct?
  - 49.How do you deep copy a struct that contains pointers?
  - 50.How do you create a method that modifies struct fields using a pointer receiver?
  - 51.How do you store a pointer to a function in a struct?
  - 52.How do you initialize a pointer to an embedded struct?
  - 53.What happens when you take the address of an embedded struct?
  - 54.How do you use a struct pointer as a function argument?
  - 55.How do you use pointers with JSON marshalling and unmarshalling?
  - 56.What are the performance benefits of using struct pointers in Go?
  - 57.How do you use struct pointers in a map?
  - 58.What are the common mistakes when using pointers with structs?
  - 59.How does Go handle struct memory alignment with pointers?
  - 60.How do you efficiently allocate memory for a large number of structs using pointers?
- 

## **Advanced (101-150) - Pointer Data Type in Go**

### **101-120: Complex Pointer Manipulations**

- 101.Implement a function that swaps two integers using pointers.
- 102.Implement a function that reverses a string using pointers.
- 103.Implement a function that reverses a slice using pointers.
- 104.Implement a function that finds the maximum value in an array using pointers.
- 105.Implement a function that sorts an array in ascending order using pointers.
- 106.Implement a function that modifies a struct field using a pointer.
- 107.Implement a function that converts a pointer to a struct into JSON.
- 108.Implement a function that calculates the factorial of a number using pointers.
- 109.Implement a function that creates a linked list using pointers.
- 110.Implement a function that traverses a linked list using pointers.
- 111.Implement a function that reverses a linked list using pointers.
- 112.Implement a function that finds a cycle in a linked list using pointers.
- 113.Implement a function that deep copies a struct containing nested pointers.
- 114.Implement a function that checks for memory leaks when using pointers.
- 115.Implement a function that manages memory allocation for a large dataset using pointers.
- 116.Implement a function that simulates pointer arithmetic using slices.
- 117.Implement a function that uses unsafe pointers to modify memory.
- 118.Implement a function that benchmarks performance differences between pointers and values.
- 119.Implement a function that dynamically resizes a slice using pointers.
- 120.Implement a function that converts a raw byte slice into a pointer and modifies its content.

## Basic Questions (1-100) - Interface Data Type in Go

### 1-20: Interface Basics

1. What is an `interface` in Go?
2. How do you define an interface in Go?
3. How do you implement an interface in Go?
4. Can a struct implement multiple interfaces?
5. What happens if a struct does not implement all methods of an interface?
6. How do you declare an empty interface?
7. What is the purpose of an empty interface (`interface{}`)?
8. Can an interface contain fields? Why or why not?
9. How do you check if a variable implements an interface?
10. How do you store a struct inside an interface?
11. What is the zero value of an interface?
12. How do you check if an interface variable is `nil`?
13. Can you use `==` to compare interface values?
14. What is the difference between an interface and a struct?
15. How do you use an interface as a function parameter?
16. How do you return an interface from a function?
17. How do you type assert an interface to a concrete type?
18. What happens if type assertion fails?
19. How do you use a type switch with interfaces?
20. What is the difference between a type assertion and a type switch?

### 21-40: Interface Implementations and Methods

21. How do you implement an interface using a pointer receiver?
22. How do you implement an interface using a value receiver?
23. What is the difference between implementing an interface with a pointer vs. a value receiver?
24. What happens if you pass a struct with a pointer receiver to an interface?
25. How do you create a method that modifies struct fields through an interface?
26. Can a method with a value receiver satisfy an interface requiring a pointer receiver?
27. Can a method with a pointer receiver satisfy an interface requiring a value receiver?
28. How do you use interfaces to achieve polymorphism in Go?
29. What is the `io.Writer` interface and how is it used?
30. How do you implement the `fmt.Stringer` interface?
31. How do you implement a custom error type using the `error` interface?
32. What is the `Reader` interface in Go?
33. What is the `Writer` interface in Go?
34. How do you implement both `Reader` and `Writer` interfaces in a struct?
35. How do you create an interface that extends another interface?
36. Can you embed one interface inside another?
37. How do you check if a type satisfies an interface at compile time?

- 38.What are implicit interfaces in Go?
- 39.How does Go's interface implementation differ from Java's and C++'s?
- 40.Can you implement an interface without explicitly stating it?

#### **41-60: Advanced Interface Concepts**

- 41.How does Go resolve interface method calls at runtime?
  - 42.What is the underlying type of an interface variable?
  - 43.How does Go store interface values in memory?
  - 44.What is an interface table (itab) in Go?
  - 45.What happens when an interface variable holds a `nil` concrete type?
  - 46.Why does an interface containing `nil` concrete value not compare equal to `nil`?
  - 47.What is interface composition?
  - 48.How does embedding multiple interfaces work?
  - 49.What are the advantages of using interfaces in Go?
  - 50.What are the disadvantages of using interfaces in Go?
  - 51.How does the Go runtime determine if a type implements an interface?
  - 52.How does reflection (`reflect` package) work with interfaces?
  - 53.How do you extract the concrete type of an interface using reflection?
  - 54.How do you modify the concrete value inside an interface using reflection?
  - 55.What is the difference between `interface{}` and `any` in Go 1.18?
  - 56.How do you safely convert `interface{}` to a specific type?
  - 57.How do you avoid runtime panics when using type assertions?
  - 58.What is the empty interface (`interface{}`) used for in Go standard libraries?
  - 59.How do you use interfaces to create flexible APIs?
  - 60.What is the difference between an empty interface and a generic type in Go 1.18?
- 

### **Advanced (101-150) - Interface Data Type in Go**

#### **101-120: Complex Interface Implementations**

- 101.Implement an interface for a simple `Shape` type with `Area()` and `Perimeter()` methods.
- 102.Implement an interface for a `Vehicle` type with methods for `Start()` and `Stop()`.
- 103.Implement an interface for a `Database` type with methods `Connect()` and `Query()`.
- 104.Implement an interface that supports multiple implementations (e.g., `Animal` interface with `Speak()` method).
- 105.Implement a function that takes an interface as a parameter and dynamically calls its methods.
- 106.Implement a type switch that handles multiple interface implementations differently.
- 107.Implement a function that checks if an interface is empty (`nil`).
- 108.Implement an interface for logging with different output formats (`JSON`, `Plain Text`).
- 109.Implement an interface that reads from different data sources (e.g., file, network, database).
- 110.Implement a `Reader` and `Writer` interface together in a struct.
- 111.Implement a function that accepts `interface{}` and safely converts it to different types.

112. Implement an interface for a caching system with `Set()` and `Get()` methods.
113. Implement an interface for a `Queue` data structure.
114. Implement an interface for a plugin system where different modules implement different behaviors.
115. Implement an interface that supports dependency injection for unit testing.
116. Implement an interface for processing different file types (CSV, JSON, XML).
117. Implement a function that filters values from a slice using an interface.
118. Implement an interface that allows different payment methods (Credit Card, PayPal, Bitcoin).
119. Implement a function that takes a list of interfaces and performs a common operation on them.
120. Implement a function that compares two interface values and determines if they are deeply equal.

## Basic Questions (1-100) - Array Data Type in Go

### 1-20: Array Basics

1. What is an array in Go?
2. How do you declare an array in Go?
3. What is the default value of an array in Go?
4. How do you initialize an array with specific values?
5. How do you access elements of an array?
6. How do you find the length of an array?
7. Can you change the size of an array after declaration? Why or why not?
8. What happens if you try to access an index out of range?
9. How do you iterate over an array using a `for` loop?
10. How do you iterate over an array using a `range` loop?
11. Can you use `append()` with arrays? Why or why not?
12. How do you copy one array to another?
13. What is the difference between an array and a slice?
14. How do you declare a multi-dimensional array in Go?
15. How do you initialize a multi-dimensional array?
16. How do you access elements in a multi-dimensional array?
17. How do you loop through a multi-dimensional array?
18. How do you check if an array contains a specific value?
19. How do you compare two arrays for equality?
20. How do you pass an array to a function?

### 21-40: Advanced Array Operations

21. How do you return an array from a function?
22. Can you use an array as a function return type?
23. How do you pass an array by reference?



- 24.How do you modify an array inside a function?
- 25.What is the performance difference between passing an array and passing a slice?
- 26.How do you reverse an array in Go?
- 27.How do you sort an array in ascending order?
- 28.How do you sort an array in descending order?
- 29.How do you remove duplicates from an array?
- 30.How do you merge two arrays into a new array?
- 31.How do you find the maximum value in an array?
- 32.How do you find the minimum value in an array?
- 33.How do you find the sum of all elements in an array?
- 34.How do you find the average of an array's elements?
- 35.How do you shift elements in an array to the left?
- 36.How do you shift elements in an array to the right?
- 37.How do you swap two elements in an array?
- 38.How do you search for an element in an unsorted array?
- 39.How do you search for an element in a sorted array using binary search?
- 40.How do you implement a stack using an array?

#### **41-60: Multi-Dimensional Arrays and Edge Cases**

- 41.What is a three-dimensional array in Go?
- 42.How do you initialize a three-dimensional array?
- 43.How do you iterate over a three-dimensional array?
- 44.How do you convert a multi-dimensional array to a slice?
- 45.How do you find the sum of all elements in a multi-dimensional array?
- 46.How do you flatten a multi-dimensional array into a one-dimensional array?
- 47.How do you rotate a 2D array (matrix) by 90 degrees?
- 48.How do you check if a matrix is symmetric?
- 49.How do you check if a matrix is an identity matrix?
- 50.How do you multiply two matrices in Go?
- 51.What happens when you assign one array to another?
- 52.Can you pass part of an array to a function?
- 53.What happens if you modify an array inside a function?
- 54.Can an array store pointers? How?
- 55.Can you create an array of interfaces?
- 56.How do you create an array of functions?
- 57.How do you convert an array to a slice?
- 58.How do you convert a slice back to an array?
- 59.What happens when you convert an array to a slice?
- 60.How does Go handle memory allocation for arrays?

---

## **Advanced (101-150) - Array Data Type in Go**

### **101-120: Complex Array Operations**

- 101.Implement a function that finds the second largest element in an array.

102. Implement a function that rotates an array to the left by k positions.
103. Implement a function that rotates an array to the right by k positions.
104. Implement a function that removes all even numbers from an array.
105. Implement a function that removes all odd numbers from an array.
106. Implement a function that counts the occurrences of each element in an array.
107. Implement a function that finds the most frequently occurring element in an array.
108. Implement a function that finds the intersection of two arrays.
109. Implement a function that finds the union of two arrays.
110. Implement a function that finds the difference between two arrays.
111. Implement a function that checks if two arrays are permutations of each other.
112. Implement a function that checks if an array is a palindrome.
113. Implement a function that finds the subarray with the maximum sum.
114. Implement a function that finds the longest increasing subsequence in an array.
115. Implement a function that finds the longest common subsequence of two arrays.
116. Implement a function that shuffles an array randomly.
117. Implement a function that partitions an array around a pivot.
118. Implement a function that merges overlapping intervals in an array.
119. Implement a function that generates all subsets of an array.
120. Implement a function that finds the majority element in an array.

## **Basic Questions (1-100) - Slice Data Type in Go**

### **1-20: Slice Basics**

1. What is a slice in Go?
2. How do you declare a slice in Go?
3. What is the difference between an array and a slice?
4. How do you initialize a slice with values?
5. How do you create a slice using the `make( )` function?
6. What is the default value of a slice?
7. How do you access elements in a slice?
8. How do you get the length of a slice?
9. How do you get the capacity of a slice?
10. What happens if you access an index out of range in a slice?
11. How do you iterate over a slice using a `for` loop?
12. How do you iterate over a slice using a `range` loop?
13. How do you append an element to a slice?
14. How do you append multiple elements to a slice?
15. How does Go handle slice resizing when using `append( )`?
16. How do you copy one slice into another?
17. What happens when you assign one slice to another?
18. How do you create a slice from an existing array?
19. What happens when you modify an element of a slice that is backed by an array?
20. How do you create a slice from another slice?

## **21-40: Slice Manipulations**

- 21.How do you remove an element from a slice?
- 22.How do you remove an element at a specific index in a slice?
- 23.How do you remove the first element of a slice?
- 24.How do you remove the last element of a slice?
- 25.How do you insert an element at a specific position in a slice?
- 26.How do you merge two slices into a new slice?
- 27.How do you check if a slice contains a specific element?
- 28.How do you find the index of an element in a slice?
- 29.How do you reverse a slice?
- 30.How do you sort a slice in ascending order?
- 31.How do you sort a slice in descending order?
- 32.How do you shuffle a slice randomly?
- 33.What happens when you slice a slice beyond its capacity?
- 34.How do you create a deep copy of a slice?
- 35.How do you create a slice of structs?
- 36.How do you pass a slice to a function?
- 37.How do you return a slice from a function?
- 38.How do you check if two slices are equal?
- 39.How do you find the maximum element in a slice?
- 40.How do you find the minimum element in a slice?

## **41-60: Advanced Slice Concepts**

- 41.How does Go handle memory allocation for slices?
  - 42.What is the relationship between slice capacity and memory allocation?
  - 43.What happens when a slice grows beyond its allocated capacity?
  - 44.How do you manually increase the capacity of a slice?
  - 45.How do you efficiently remove elements from a large slice?
  - 46.What are the performance implications of modifying a slice inside a function?
  - 47.How do you prevent accidental modification of a slice passed to a function?
  - 48.How do you efficiently split a slice into multiple smaller slices?
  - 49.How do you rotate a slice to the left?
  - 50.How do you rotate a slice to the right?
  - 51.How do you implement a queue using a slice?
  - 52.How do you implement a stack using a slice?
  - 53.How do you efficiently filter elements in a slice?
  - 54.How do you check if a slice has duplicate elements?
  - 55.How do you find the intersection of two slices?
  - 56.How do you find the union of two slices?
  - 57.How do you find the difference between two slices?
  - 58.How do you create a slice of pointers?
  - 59.How do you use slices with concurrency?
  - 60.How does garbage collection affect slices in Go?
-

## **Advanced (101-150) - Slice Data Type in Go**

### **101-120: Complex Slice Operations**

101. Implement a function that removes all even numbers from a slice.
102. Implement a function that removes all odd numbers from a slice.
103. Implement a function that removes duplicate elements from a slice.
104. Implement a function that finds the most frequent element in a slice.
105. Implement a function that rotates a slice k positions to the left.
106. Implement a function that rotates a slice k positions to the right.
107. Implement a function that partitions a slice around a pivot element.
108. Implement a function that merges overlapping intervals in a slice.
109. Implement a function that finds the longest increasing subsequence in a slice.
110. Implement a function that finds the longest common subsequence of two slices.
111. Implement a function that shuffles a slice randomly.
112. Implement a function that finds the subarray with the maximum sum (Kadane's Algorithm).
113. Implement a function that finds the majority element in a slice.
114. Implement a function that generates all subsets of a slice.
115. Implement a function that implements quicksort using slices.
116. Implement a function that implements mergesort using slices.
117. Implement a function that groups anagrams from a slice of strings.
118. Implement a function that finds the kth smallest element in a slice.
119. Implement a function that merges K sorted slices.
120. Implement a function that converts a slice of integers into a string representation.

## **Basic Questions (1-100) - Struct Data Type in Go**

### **1-20: Struct Basics**

1. What is a struct in Go?
2. How do you declare a struct in Go?
3. How do you create an instance of a struct?
4. How do you initialize a struct with values?
5. What is the zero value of a struct in Go?
6. How do you access and modify struct fields?
7. Can struct fields have different data types?
8. How do you create an anonymous struct in Go?
9. How do you compare two structs?
10. Can a struct contain another struct? How?
11. How do you create a slice of structs?
12. How do you iterate over a slice of structs?
13. How do you check if a struct field is empty?

- 14.Can you create a struct without field names?
- 15.How do you create a pointer to a struct?
- 16.How do you access struct fields using a pointer?
- 17.What happens when you assign one struct to another?
- 18.How do you pass a struct to a function?
- 19.How do you return a struct from a function?
- 20.How do you define methods for a struct?

#### **21-40: Struct Methods and Advanced Usage**

- 21.How do you create a method for a struct using a value receiver?
- 22.How do you create a method for a struct using a pointer receiver?
- 23.What is the difference between value and pointer receivers in struct methods?
- 24.When should you use a pointer receiver instead of a value receiver?
- 25.Can struct methods modify struct fields?
- 26.How do you implement an interface using a struct?
- 27.Can a struct implement multiple interfaces?
- 28.How do you embed one struct inside another?
- 29.What is struct composition, and how is it different from inheritance?
- 30.How do you override an embedded struct's method?
- 31.How do you create a constructor function for a struct?
- 32.How do you add default values to struct fields?
- 33.How do you use struct tags in Go?
- 34.How do you parse struct tags?
- 35.How do you serialize a struct into JSON?
- 36.How do you deserialize JSON into a struct?
- 37.How do you handle missing or extra JSON fields when unmarshaling?
- 38.How do you serialize a struct into XML?
- 39.How do you deserialize XML into a struct?
- 40.How do you implement a custom JSON marshaling function for a struct?

#### **41-60: Working with Structs in Different Contexts**

- 41.How do you copy a struct in Go?
- 42.How do you deep copy a struct?
- 43.How do you compare two structs for equality?
- 44.How do you sort a slice of structs by a field?
- 45.How do you filter a slice of structs based on a condition?
- 46.How do you find an element in a slice of structs?
- 47.How do you check if a slice of structs contains a specific struct?
- 48.How do you group structs by a specific field?
- 49.How do you count the occurrences of a struct field in a slice?
- 50.How do you store structs in a map?
- 51.Can you use a struct as a key in a map? Why or why not?
- 52.How do you use struct pointers in a map?
- 53.How do you sort a map of structs by a specific field?
- 54.How do you use reflection with structs?

- 55.How do you list all the fields of a struct using reflection?
  - 56.How do you change struct field values using reflection?
  - 57.How do you determine the type of a struct at runtime?
  - 58.How do you check if a struct implements an interface?
  - 59.How do you create a struct that supports deep equality checks?
  - 60.How do you prevent struct modifications by making fields unexported?
- 

## **Advanced (101-150) - Struct Data Type in Go**

### **101-120: Complex Struct Operations**

- 101.Implement a function that deep copies a struct.
- 102.Implement a function that converts a struct to a map.
- 103.Implement a function that converts a map to a struct.
- 104.Implement a function that finds the difference between two structs.
- 105.Implement a function that merges two structs.
- 106.Implement a function that validates struct fields based on tags.
- 107.Implement a function that generates a JSON schema from a struct.
- 108.Implement a function that compresses a struct into a binary format.
- 109.Implement a function that decompresses a binary format into a struct.
- 110.Implement a function that masks sensitive struct fields when printing.
- 111.Implement a function that converts a struct to a CSV row.
- 112.Implement a function that reads CSV data into a struct slice.
- 113.Implement a function that logs struct changes over time.
- 114.Implement a function that calculates the hash of a struct.
- 115.Implement a function that serializes a struct into YAML.
- 116.Implement a function that deserializes YAML into a struct.
- 117.Implement a function that converts a struct to a SQL query.
- 118.Implement a function that scans a SQL row into a struct.
- 119.Implement a function that detects if a struct contains circular references.
- 120.Implement a function that dynamically fills a struct from an external config file.

## **Basic Questions (1-100) - Map Data Type in Go**

### **1-20: Map Basics**

1. What is a map in Go?
2. How do you declare a map in Go?
3. How do you initialize a map using `make( )`?
4. How do you initialize a map using a literal?
5. What types can be used as keys in a map?
6. What types can be used as values in a map?
7. What happens if you access a key that doesn't exist?
8. How do you check if a key exists in a map?

9. How do you add a key-value pair to a map?
10. How do you update a value in a map?
11. How do you delete a key-value pair from a map?
12. How do you find the number of elements in a map?
13. How do you iterate over a map using a `for` loop?
14. How do you iterate over a map using `range`?
15. How do you copy a map?
16. Can you compare two maps directly? Why or why not?
17. How do you clear all elements from a map?
18. How do you retrieve all keys from a map?
19. How do you retrieve all values from a map?
20. How do you convert a map to a slice?

#### **21-40: Working with Different Map Key and Value Types**

21. How do you use a struct as a key in a map?
22. How do you use a struct as a value in a map?
23. How do you use a slice as a value in a map?
24. Why can't you use a slice as a key in a map?
25. How do you store a map inside another map?
26. How do you create a map of slices?
27. How do you create a map of maps?
28. How do you find a key with the highest value in a map?
29. How do you find a key with the lowest value in a map?
30. How do you find all keys that contain a specific value?
31. How do you merge two maps?
32. How do you deep copy a map with nested structures?
33. How do you remove duplicate values from a map?
34. How do you implement a frequency counter using a map?
35. How do you swap keys and values in a map?
36. How do you reverse a map?
37. How do you use a map to group elements by a certain property?
38. How do you use a map to implement a cache?
39. How do you create a map with a default value?
40. How do you implement a map with a limited size?

#### **41-60: Map Performance and Internals**

41. How are maps implemented internally in Go?
42. What is the time complexity of accessing an element in a map?
43. What is the time complexity of inserting an element in a map?
44. What is the time complexity of deleting an element from a map?
45. What happens when a map grows beyond its capacity?
46. How does Go handle hash collisions in maps?
47. What happens when a key is removed from a map?
48. How does Go determine the order of iteration in maps?
49. Why does the iteration order of a map change between executions?

- 50.What is the difference between a nil map and an empty map?
  - 51.Can you use a `sync.Mutex` with a map to make it thread-safe?
  - 52.What is `sync.Map` and how does it differ from a regular map?
  - 53.When should you use `sync.Map` instead of a regular map?
  - 54.How do you safely update a shared map in concurrent code?
  - 55.How does garbage collection affect maps in Go?
  - 56.What is the impact of large keys on map performance?
  - 57.How do you optimize memory usage in a large map?
  - 58.How do you detect memory leaks in maps?
  - 59.How do you efficiently delete many elements from a large map?
  - 60.How do you limit the memory usage of a large map?
- 

## Advanced (101-150) - Map Data Type in Go

### 101-120: Complex Map Operations

- 101.Implement a function that finds the most frequently occurring element in a map.
- 102.Implement a function that finds the least frequently occurring element in a map.
- 103.Implement a function that finds the top N elements with the highest values in a map.
- 104.Implement a function that finds the sum of all values in a map.
- 105.Implement a function that groups words by their first letter using a map.
- 106.Implement a function that groups numbers by their remainder when divided by a given number.
- 107.Implement a function that counts the occurrences of each letter in a string using a map.
- 108.Implement a function that checks if two maps are equal.
- 109.Implement a function that finds all unique values in a map.
- 110.Implement a function that converts a map to JSON.
- 111.Implement a function that loads JSON data into a map.
- 112.Implement a function that removes keys with zero values from a map.
- 113.Implement a function that finds the longest key in a map.
- 114.Implement a function that finds the shortest key in a map.
- 115.Implement a function that reverses a map of string keys and integer values.
- 116.Implement a function that builds an index from a list of words using a map.
- 117.Implement a function that creates a lookup table using a map.
- 118.Implement a function that generates a histogram using a map.
- 119.Implement a function that implements a simple key-value store with expiration.
- 120.Implement a function that finds the common keys between two maps.

## Basic Questions (1-100) - Pointer Data Type in Go

### 1-20: Pointer Basics

1. What is a pointer in Go?
2. How do you declare a pointer in Go?



3. How do you assign a value to a pointer?
4. What does the `*` operator do in Go when used with a pointer?
5. What does the `&` operator do in Go?
6. What is the zero value of a pointer?
7. How do you check if a pointer is `nil`?
8. How do you modify the value of a variable through a pointer?
9. How do you pass a pointer to a function?
10. How do you return a pointer from a function?
11. Can you have a pointer to a pointer in Go?
12. How do you create a pointer to a struct?
13. How do you access struct fields using a pointer?
14. What happens if you dereference a `nil` pointer?
15. How do you copy a pointer?
16. Can you compare two pointers?
17. What happens when you assign one pointer to another?
18. How do you allocate memory dynamically for a pointer?
19. What is the difference between `new()` and `make()` in Go?
20. How do you print the memory address stored in a pointer?

#### **21-40: Working with Pointers in Different Contexts**

21. How do you use a pointer with an array?
22. How do you use a pointer with a slice?
23. How do you use a pointer with a map?
24. How do you use a pointer with a struct containing another struct?
25. What is a pointer receiver in a struct method?
26. How do you modify a struct field inside a method using a pointer receiver?
27. How do you implement a setter function using a pointer receiver?
28. How do you return a pointer to a struct from a function?
29. How do you prevent accidental modifications to struct fields using pointers?
30. How do you create a function that returns a pointer to an integer?
31. How do you create a function that swaps two integers using pointers?
32. How do you pass a slice to a function using a pointer?
33. How do you pass a map to a function using a pointer?
34. Why doesn't Go allow pointer arithmetic like C?
35. Can a function return multiple pointers?
36. What is the difference between passing a value and passing a pointer to a function?
37. How do you use pointers with variadic functions?
38. How do you safely allocate a large struct on the heap using a pointer?
39. What happens when you assign `nil` to a pointer?
40. How do you safely free memory allocated by a pointer in Go?

#### **41-60: Pointers and Concurrency**

41. How do you use pointers with goroutines?
42. What are the dangers of sharing a pointer between multiple goroutines?
43. How do you avoid race conditions when using pointers?

- 44.How do you use `sync.Mutex` to protect a pointer?
  - 45.How do you use `sync/atomic` to safely update a pointer?
  - 46.What is `unsafe.Pointer` and when should you use it?
  - 47.How do you convert a `uintptr` to a pointer?
  - 48.How do you convert a pointer to `uintptr`?
  - 49.How do you use `unsafe.Pointer` to manipulate memory?
  - 50.Why is `unsafe.Pointer` considered dangerous?
  - 51.How does Go's garbage collector handle pointers?
  - 52.How do you prevent memory leaks when using pointers?
  - 53.How do you use pointers to implement a linked list?
  - 54.How do you use pointers to implement a tree structure?
  - 55.How do you use pointers to implement a stack?
  - 56.How do you use pointers to implement a queue?
  - 57.How do you use pointers to optimize memory usage in a large struct?
  - 58.How do you use pointers to efficiently modify a large data structure?
  - 59.How do you use pointers to pass large slices efficiently?
  - 60.What are the common pitfalls of using pointers in Go?
- 

## **Advanced (101-150) - Pointer Data Type in Go**

### **101-120: Complex Pointer Operations**

- 101.Implement a function that reverses an array using pointers.
- 102.Implement a function that swaps two struct fields using pointers.
- 103.Implement a function that modifies a string in place using pointers.
- 104.Implement a function that initializes a slice using a pointer.
- 105.Implement a function that finds the maximum value in an array using pointers.
- 106.Implement a function that implements a doubly linked list using pointers.
- 107.Implement a function that removes duplicates from a linked list using pointers.
- 108.Implement a function that merges two sorted linked lists using pointers.
- 109.Implement a function that detects a cycle in a linked list using pointers.
- 110.Implement a function that finds the middle of a linked list using pointers.
- 111.Implement a function that reverses a linked list using pointers.
- 112.Implement a function that copies a tree structure using pointers.
- 113.Implement a function that serializes a tree into a slice using pointers.
- 114.Implement a function that deserializes a slice into a tree using pointers.
- 115.Implement a function that calculates the sum of a binary tree using pointers.
- 116.Implement a function that performs an in-order traversal of a tree using pointers.
- 117.Implement a function that performs a pre-order traversal of a tree using pointers.
- 118.Implement a function that performs a post-order traversal of a tree using pointers.
- 119.Implement a function that efficiently resize a large slice using pointers.
- 120.Implement a function that optimizes a large struct by using pointers instead of values.

## Basic Questions (1-100) - Interface Data Type in Go

### 1-20: Interface Basics

1. What is an interface in Go?
2. How do you define an interface in Go?
3. How do you implement an interface in Go?
4. Can a struct implement multiple interfaces?
5. How do you check if a struct implements an interface?
6. How do you use an interface as a function parameter?
7. How do you use an interface as a function return type?
8. What happens if a struct does not implement all methods of an interface?
9. Can interfaces have fields in Go? Why or why not?
10. How do you declare an empty interface?
11. What is the purpose of an empty interface (`interface{}`)?
12. How do you type assert an interface to a concrete type?
13. What happens if a type assertion fails?
14. How do you handle a failed type assertion safely?
15. How do you use a type switch with an interface?
16. How does Go determine if a type satisfies an interface?
17. How do you create a slice of interfaces?
18. Can an interface contain another interface?
19. What is a nil interface?
20. How do you check if an interface is nil?

### 21-40: Interface Usage and Practical Scenarios

21. How do you use an interface to achieve polymorphism in Go?
22. How do you store multiple data types in a slice using an interface?
23. How do you define a method that accepts multiple interface types?
24. How do you return different struct types from a function using an interface?
25. How do you compare two interface values?
26. How do you use an interface to mock behavior in tests?
27. What is the difference between an interface and a struct?
28. Can an interface contain private methods?
29. How do you convert a concrete type to an interface?
30. How do you convert an interface back to its original type?
31. How do you dynamically determine the type of an interface value?
32. How do you use interfaces in Go's standard library?
33. What is the `io.Reader` interface and how is it used?
34. What is the `io.Writer` interface and how is it used?
35. How do you implement a custom `Stringer` interface?
36. How do you implement a custom `error` interface?
37. How do you use an interface to define a database driver?
38. How do you use an interface to define a logger in Go?
39. How do you use an interface to create a plugin system in Go?
40. What is an interface embedding and how does it work?

## 41-60: Interface Internals and Performance

41. How does Go store an interface value internally?
  42. What is the difference between static typing and dynamic typing in Go?
  43. How does method resolution work in Go interfaces?
  44. What is interface dispatch in Go?
  45. What is an interface table (`itab`) in Go?
  46. How does Go handle method calls on an interface?
  47. What is the cost of using interfaces in Go?
  48. How does interface type assertion affect performance?
  49. How does interface conversion impact memory allocation?
  50. Why should you avoid using `interface{}` excessively?
  51. How does Go's garbage collector treat interfaces?
  52. Can you use reflection with interfaces?
  53. How do you use `reflect.TypeOf()` with interfaces?
  54. How do you use `reflect.ValueOf()` with interfaces?
  55. How do you determine if an interface holds a specific type using reflection?
  56. How does Go optimize interface method calls?
  57. What is the difference between an interface value containing `nil` vs. a `nil` interface?
  58. How do you measure the memory footprint of an interface?
  59. When should you use an interface instead of a struct?
  60. When should you use a struct instead of an interface?
- 

## Advanced (101-150) - Interface Data Type in Go

### 101-120: Complex Interface Implementations

101. Implement a simple logging interface with multiple loggers (console, file).
102. Implement a database interface that supports multiple database types.
103. Implement an interface for sending messages via email and SMS.
104. Implement a sorting function that works with different types using interfaces.
105. Implement a caching interface with in-memory and file-based implementations.
106. Implement a payment processing interface with PayPal and Stripe implementations.
107. Implement an interface for an API client that supports multiple backends.
108. Implement an interface for reading data from different sources (file, database).
109. Implement an event listener interface with multiple event handlers.
110. Implement a middleware system using interfaces.
111. Implement a job queue interface with different backends (in-memory, Redis).
112. Implement an interface that supports dependency injection.
113. Implement a configuration loader interface that reads from JSON and YAML.
114. Implement an interface for handling different authentication methods.
115. Implement a plugin system using interfaces.
116. Implement an interface for processing image files (JPEG, PNG, GIF).
117. Implement a rate limiter using an interface.
118. Implement an interface for parsing different data formats (CSV, JSON, XML).
119. Implement an interface for processing different types of HTTP requests.

120. Implement an observer pattern using interfaces.

## **Basic Questions (1-100) - Channels in Go**

### **1-20: Channel Basics**

1. What is a channel in Go?
2. How do you declare a channel in Go?
3. How do you initialize a channel?
4. What is the difference between a buffered and an unbuffered channel?
5. How do you send a value to a channel?
6. How do you receive a value from a channel?
7. What happens if you send data to a nil channel?
8. What happens if you receive data from a nil channel?
9. How do you check if a channel is open or closed?
10. How do you close a channel?
11. What happens if you send data to a closed channel?
12. What happens if you receive data from a closed channel?
13. Can you reopen a closed channel? Why or why not?
14. What is a deadlock in Go channels?
15. How do you avoid deadlocks when working with channels?
16. What happens if there are no goroutines ready to receive from a channel?
17. What happens if there are no goroutines ready to send to a channel?
18. Can channels be passed as function arguments?
19. How do you create a bidirectional channel?
20. How do you create a send-only or receive-only channel?

### **21-40: Buffered and Unbuffered Channels**

21. What is the default buffer size of an unbuffered channel?
22. How do you declare a buffered channel?
23. What is the behavior of a buffered channel when it is full?
24. What happens when a buffered channel is empty and a receive operation is performed?
25. How do you check the capacity of a buffered channel?
26. How do you check the length of a buffered channel?
27. What is the difference in performance between buffered and unbuffered channels?
28. When should you use a buffered channel instead of an unbuffered channel?
29. How do you handle blocking issues in buffered channels?
30. How do you empty a buffered channel safely?
31. Can a buffered channel cause a deadlock? Why or why not?
32. How do you handle a full buffered channel without blocking?
33. How do you use a loop to receive all values from a buffered channel?
34. How do you use a buffered channel to implement a task queue?
35. How do you determine the optimal buffer size for a channel?
36. Can a buffered channel have zero capacity?
37. What happens when you close a buffered channel with unread values?

- 38.How do you drain a buffered channel before closing it?
- 39.Can you dynamically resize a buffered channel? Why or why not?
- 40.How do you use a buffered channel to implement a worker pool?

#### **41-60: Channel Communication Patterns**

- 41.What is the `range` keyword used for in channels?
- 42.How do you iterate over a channel using `range`?
- 43.How do you use channels for signaling between goroutines?
- 44.How do you use a channel as a semaphore?
- 45.How do you use channels to implement a producer-consumer pattern?
- 46.How do you use a channel to control the execution order of goroutines?
- 47.How do you use multiple channels to communicate between multiple goroutines?
- 48.How do you ensure that a channel is read from before sending more data?
- 49.What happens when multiple goroutines try to send data to the same channel?
- 50.What happens when multiple goroutines try to receive data from the same channel?
- 51.How do you synchronize multiple goroutines using channels?
- 52.How do you implement a timeout for a channel operation?
- 53.What is the `select` statement used for in channels?
- 54.How do you use `select` to listen to multiple channels?
- 55.What happens if multiple `case` conditions in a `select` statement are ready?
- 56.What happens if no `case` conditions in a `select` statement are ready?
- 57.How do you use the `default` case in a `select` statement?
- 58.How do you prevent a `select` statement from blocking indefinitely?
- 59.How do you use channels to implement an event-driven system?
- 60.How do you use a channel to implement a pipeline?

#### **61-80: Channel Synchronization and Performance**

- 61.How do you use channels for goroutine synchronization?
- 62.How do you use channels instead of `sync.WaitGroup`?
- 63.How do you implement a rate limiter using channels?
- 64.How do you prevent a channel from leaking memory?
- 65.What are common mistakes developers make when using channels?
- 66.How do you handle slow consumers in a channel system?
- 67.How do you avoid race conditions when using channels?
- 68.How do you benchmark the performance of channels?
- 69.What is the impact of channel buffering on memory usage?
- 70.How does garbage collection handle channels?
- 71.How do you use channels to implement a timeout for a long-running operation?
- 72.How do you limit the number of goroutines using a channel?
- 73.How do you implement a thread-safe counter using channels?
- 74.How do you prevent high contention on a single channel?
- 75.How do you implement message prioritization using channels?
- 76.How do you merge multiple channels into one?
- 77.How do you split one channel into multiple channels?
- 78.How do you ensure that a channel is closed exactly once?

- 79.How do you implement an actor model using channels?
- 80.What are the trade-offs of using channels for synchronization instead of mutexes?
- 

## **Advanced (101-150) - Channels in Go**

### **101-120: Complex Channel Implementations**

- 101.Implement a fan-in pattern using channels.
- 102.Implement a fan-out pattern using channels.
- 103.Implement a pipeline using channels.
- 104.Implement a rate limiter using a buffered channel.
- 105.Implement a job queue using a buffered channel.
- 106.Implement a timeout for a function call using a channel.
- 107.Implement a traffic light system using channels.
- 108.Implement a concurrent map using channels.
- 109.Implement an event bus using channels.
- 110.Implement an observer pattern using channels.
- 111.Implement a bounded queue using a buffered channel.
- 112.Implement a cancellation signal using channels.
- 113.Implement an HTTP request limiter using channels.
- 114.Implement a scheduler using channels.
- 115.Implement a pub-sub system using channels.
- 116.Implement a round-robin dispatcher using channels.
- 117.Implement an auto-scaling worker pool using channels.
- 118.Implement a message broker using channels.
- 119.Implement a priority queue using multiple channels.
- 120.Implement a retry mechanism using channels.

## **Basic Questions (1-100) - Goroutines in Go**

### **1-20: Goroutine Basics**

- 1. What is a goroutine in Go?
- 2. How do you start a goroutine?
- 3. How does a goroutine differ from an OS thread?
- 4. What happens when a main function exits while goroutines are still running?
- 5. How do you prevent a program from exiting before all goroutines complete?
- 6. How do you wait for a goroutine to finish?
- 7. How do you pass arguments to a goroutine?
- 8. What happens if you pass a pointer to a variable in a goroutine?
- 9. How does Go's runtime scheduler handle goroutines?
- 10.Can a goroutine return a value? Why or why not?
- 11.How do you handle return values from a goroutine?
- 12.Can a goroutine panic? If so, how do you recover from it?

13. How do you limit the number of goroutines running simultaneously?
14. What happens when a goroutine blocks on a channel?
15. What is the difference between `go func()` and calling `func()` normally?
16. Can a function execute both synchronously and asynchronously in Go?
17. How do you stop a running goroutine?
18. Why should you avoid using `go` on anonymous functions that reference loop variables?
19. How do you pass large data structures to goroutines efficiently?
20. Can a goroutine be executed multiple times?

## **21-40: Goroutine Management**

21. How do you synchronize multiple goroutines?
22. How do you use `sync.WaitGroup` to manage goroutines?
23. What is a goroutine leak?
24. How do you avoid goroutine leaks?
25. How do you detect goroutine leaks in a Go program?
26. What is the `runtime.NumGoroutine()` function used for?
27. How do you find the number of active goroutines in a Go program?
28. What is the difference between `sync.Mutex` and channels for synchronization?
29. How do you handle shared data between goroutines?
30. What is a race condition, and how can it occur in goroutines?
31. How do you debug race conditions in goroutines?
32. How does `sync/atomic` help in goroutine synchronization?
33. How do you use a `context.Context` to control goroutines?
34. How do you implement a goroutine timeout using `context.WithTimeout`?
35. How do you implement a cancellation signal for a goroutine?
36. How does Go's garbage collector interact with goroutines?
37. How do you avoid memory leaks caused by goroutines?
38. How does Go handle scheduling for a large number of goroutines?
39. How do you implement worker pools using goroutines?
40. How do you measure the execution time of a goroutine?

## **41-60: Goroutines and Channels**

41. What is the relationship between goroutines and channels?
42. How do you use a channel to synchronize goroutines?
43. How do you signal a goroutine to stop using a channel?
44. What happens if a goroutine sends data to a channel that is never read?
45. How do you prevent a goroutine from blocking on a channel operation?
46. How do you implement a fan-out pattern using goroutines?
47. How do you implement a fan-in pattern using goroutines?
48. What is the purpose of a `select` statement in goroutines?
49. How do you use `select` to handle multiple channels in goroutines?
50. How do you avoid deadlocks when using goroutines and channels?
51. How do you implement a timeout mechanism in a goroutine using channels?
52. What happens when you close a channel that is being used by a goroutine?
53. How do you handle errors in a goroutine?



- 54.How do you implement an event-driven system using goroutines?
- 55.What is a pipeline pattern in goroutines?
- 56.How do you implement a rate limiter using goroutines?
- 57.How do you merge multiple channels into a single channel using goroutines?
- 58.How do you split a single channel into multiple channels?
- 59.What is a semaphore, and how can it be implemented using goroutines?
- 60.How do you use goroutines for parallel processing?

## **61-80: Performance and Best Practices**

- 61.How do you profile goroutine execution in Go?
  - 62.How do you optimize goroutine performance?
  - 63.How do you avoid excessive goroutine creation?
  - 64.What is the impact of goroutines on CPU and memory usage?
  - 65.How do you use GOMAXPROCS to control goroutine execution?
  - 66.How does Go's runtime distribute goroutines across CPU cores?
  - 67.How do you use goroutines for I/O-bound operations?
  - 68.How do you use goroutines for CPU-bound operations?
  - 69.When should you use goroutines instead of traditional threading models?
  - 70.How does garbage collection impact goroutines?
  - 71.What are best practices for error handling in goroutines?
  - 72.How do you implement graceful shutdown for goroutines?
  - 73.How do you log events from multiple goroutines?
  - 74.How do you avoid synchronization overhead when using goroutines?
  - 75.How do you use goroutines to parallelize computations?
  - 76.How do you implement an actor model using goroutines?
  - 77.How do you ensure a goroutine is executed only once?
  - 78.How do you manage memory consumption in a high-concurrency Go program?
  - 79.How do you test functions that use goroutines?
  - 80.How do you handle panics inside goroutines?
- 

## **Advanced (101-150) - Goroutines in Go**

### **101-120: Complex Goroutine Implementations**

- 101.Implement a thread-safe counter using goroutines.
- 102.Implement a bounded queue using goroutines.
- 103.Implement a job queue using goroutines.
- 104.Implement a task scheduler using goroutines.
- 105.Implement a distributed task execution system using goroutines.
- 106.Implement an event loop using goroutines.
- 107.Implement a web scraper using goroutines.
- 108.Implement an HTTP request handler using goroutines.
- 109.Implement a load balancer using goroutines.
- 110.Implement a database connection pool using goroutines.
- 111.Implement an in-memory cache using goroutines.

- 112.Implement a priority queue using goroutines.
- 113.Implement a batch processor using goroutines.
- 114.Implement a transaction manager using goroutines.
- 115.Implement a failover mechanism using goroutines.
- 116.Implement a distributed worker pool using goroutines.
- 117.Implement a traffic analyzer using goroutines.
- 118.Implement a concurrent map using goroutines.
- 119.Implement an event-driven system using goroutines.
- 120.Implement a real-time data processing system using goroutines.

## **Basic Questions (1-100) - Wait Groups in Go**

### **1-20: WaitGroup Basics**

1. What is a `sync.WaitGroup` in Go?
2. How do you declare a `WaitGroup`?
3. How do you add a goroutine to a `WaitGroup`?
4. What does the `wg.Done()` method do?
5. What does the `wg.Wait()` method do?
6. What happens if you call `wg.Wait()` before all `wg.Done()` calls are executed?
7. What happens if you call `wg.Done()` more times than `wg.Add(n)`?
8. What happens if you forget to call `wg.Done()` in a goroutine?
9. Can a `WaitGroup` go negative? Why or why not?
- 10.How does `sync.WaitGroup` prevent the main function from exiting too early?
- 11.Can you use `WaitGroup` with multiple goroutines?
- 12.What is the default value of a `WaitGroup` counter?
- 13.What is the purpose of `wg.Add(1)` in goroutines?
- 14.Can `wg.Add(n)` be called from inside a goroutine?
- 15.What happens if you call `wg.Wait()` before calling `wg.Add(n)`?
- 16.Why should you use `defer wg.Done()` inside a goroutine?
- 17.Can you reuse a `WaitGroup` after it reaches zero?
- 18.How do you wait for all goroutines to complete before continuing execution?
- 19.What is the difference between `WaitGroup` and channels for synchronization?
- 20.Why should you avoid copying a `WaitGroup` variable?

### **21-40: WaitGroup Usage and Common Issues**

- 21.What happens if you add a negative value to `wg.Add(n)`?
- 22.Can you call `wg.Add(1)` after `wg.Wait()` has been called?
- 23.What happens if you forget to call `wg.Wait()`?
- 24.How do you use `sync.WaitGroup` inside a loop?
- 25.What happens if a goroutine panics while using `WaitGroup`?
- 26.How do you avoid a goroutine deadlock when using `WaitGroup`?

- 27.What happens if multiple goroutines call `wg.Wait()`?
- 28.How do you determine when to use `WaitGroup` versus channels?
- 29.Can `WaitGroup` be used to limit the number of running goroutines?
- 30.Why is it important to call `wg.Add(n)` before launching goroutines?
- 31.How do you debug a stuck `WaitGroup.Wait()` call?
- 32.How do you ensure that all goroutines complete execution before the main function exits?
- 33.What is the impact of calling `wg.Wait()` multiple times?
- 34.Can `sync.WaitGroup` be used with an infinite loop?
- 35.How do you implement error handling with `WaitGroup`?
- 36.What happens if a `WaitGroup` is garbage collected while still waiting?
- 37.Can `WaitGroup` be used in a recursive function?
- 38.What is the purpose of using `WaitGroup` inside an HTTP server handler?
- 39.How do you handle slow-running goroutines when using `WaitGroup`?
- 40.How do you combine `WaitGroup` with a timeout mechanism?

#### **41-60: Performance and Best Practices**

- 41.How does `sync.WaitGroup` internally work?
- 42.Is `sync.WaitGroup` thread-safe?
- 43.Does `WaitGroup` consume a lot of memory?
- 44.How efficient is `sync.WaitGroup` compared to channels for synchronization?
- 45.How do you use `WaitGroup` with a worker pool?
- 46.How do you limit the number of concurrent goroutines using `WaitGroup`?
- 47.How does `sync.WaitGroup` compare to `sync.Mutex` in functionality?
- 48.Can `sync.WaitGroup` be used to synchronize multiple independent tasks?
- 49.What is the impact of adding too many goroutines to a `WaitGroup`?
- 50.When should you avoid using `sync.WaitGroup`?
- 51.Can `sync.WaitGroup` be used in a distributed system?
- 52.How does Go optimize `sync.WaitGroup` performance?
- 53.What is the difference between `sync.WaitGroup` and `sync.Cond`?
- 54.How do you use `WaitGroup` with `sync.Once`?
- 55.How do you measure the performance of `WaitGroup`?
- 56.How do you optimize a program that heavily relies on `WaitGroup`?
- 57.Does `WaitGroup` work efficiently on multi-core systems?
- 58.What are common mistakes when using `sync.WaitGroup`?
- 59.How do you detect memory leaks in programs using `WaitGroup`?
- 60.How do you prevent excessive resource consumption when using `WaitGroup`?

#### **61-80: Advanced Synchronization Patterns**

- 61.How do you use `sync.WaitGroup` with `sync.Map`?
- 62.How do you use `sync.WaitGroup` for batch processing?
- 63.How do you combine `WaitGroup` with rate limiting?
- 64.How do you dynamically adjust the number of goroutines in a `WaitGroup`?
- 65.How do you restart a `WaitGroup` for multiple rounds of execution?

- 66.Can `sync.WaitGroup` be used to synchronize multiple dependent tasks?
  - 67.How do you prevent `WaitGroup` from blocking indefinitely?
  - 68.How do you use `WaitGroup` to coordinate database queries in parallel?
  - 69.How do you integrate `WaitGroup` with `context.Context`?
  - 70.How do you handle errors from multiple goroutines when using `WaitGroup`?
  - 71.How do you implement a job dispatcher using `WaitGroup`?
  - 72.How do you use `WaitGroup` in a microservices architecture?
  - 73.Can `sync.WaitGroup` be used to control concurrency in an API server?
  - 74.How do you handle cancellation with `sync.WaitGroup`?
  - 75.How do you combine `WaitGroup` with a message queue system?
  - 76.How do you implement backoff retry logic using `WaitGroup`?
  - 77.How do you collect results from multiple goroutines using `WaitGroup`?
  - 78.How do you use `WaitGroup` to synchronize log writing from multiple goroutines?
  - 79.How do you use `sync.WaitGroup` for parallel file processing?
  - 80.How do you gracefully terminate all goroutines using `WaitGroup`?
- 

## **Advanced (101-150) - Wait Groups in Go**

### **101-120: Complex WaitGroup Implementations**

- 101.Implement a rate limiter using `WaitGroup`.
- 102.Implement a worker pool using `WaitGroup`.
- 103.Implement a batch job processor using `WaitGroup`.
- 104.Implement a database connection manager using `WaitGroup`.
- 105.Implement a concurrent map builder using `WaitGroup`.
- 106.Implement a distributed task queue using `WaitGroup`.
- 107.Implement a multi-stage pipeline using `WaitGroup`.
- 108.Implement a priority queue using `WaitGroup`.
- 109.Implement a retry mechanism for failed goroutines using `WaitGroup`.
- 110.Implement an event-driven system using `WaitGroup`.
- 111.Implement a real-time log aggregator using `WaitGroup`.
- 112.Implement an actor model using `WaitGroup`.
- 113.Implement a rate-controlled HTTP client using `WaitGroup`.
- 114.Implement a real-time analytics system using `WaitGroup`.
- 115.Implement a high-performance task scheduler using `WaitGroup`.
- 116.Implement a transactional database writer using `WaitGroup`.
- 117.Implement a task prioritization system using `WaitGroup`.
- 118.Implement a hybrid synchronization model using `WaitGroup` and `sync.Cond`.
- 119.Implement a peer-to-peer messaging system using `WaitGroup`.
- 120.Implement a real-time monitoring system using `WaitGroup`.

# Basic Questions (1-100) - Mutex in Go

## 1-20: Mutex Basics

1. What is a `sync.Mutex` in Go?
2. How do you declare a `sync.Mutex`?
3. What is the purpose of a mutex in concurrent programming?
4. How do you lock a mutex in Go?
5. How do you unlock a mutex in Go?
6. What happens if you try to unlock a mutex that is already unlocked?
7. What happens if you try to unlock a mutex that is not locked by the current goroutine?
8. Can a mutex be locked multiple times by the same goroutine?
9. What is a deadlock in the context of a mutex?
10. What is the difference between `sync.Mutex` and `sync.RWMutex`?
11. How do you prevent deadlocks when using a mutex?
12. What happens if a goroutine panics while holding a mutex?
13. Why is it recommended to use `defer mu.Unlock()` immediately after `mu.Lock()`?
14. Can a `sync.Mutex` be copied? Why or why not?
15. Is `sync.Mutex` thread-safe?
16. How do you avoid unnecessary blocking with mutexes?
17. What are the performance implications of using a mutex?
18. When should you use a mutex instead of a channel?
19. Can a `sync.Mutex` be used to synchronize access to multiple shared resources?
20. How does Go handle mutex contention internally?

## 21-40: Mutex Usage in Goroutines

21. Can you use a mutex in the main goroutine?
22. How do you use a mutex inside a struct?
23. How do you use `sync.Mutex` with a map?
24. How do you prevent multiple goroutines from writing to the same variable simultaneously?
25. Can you use a mutex to protect a slice?
26. What happens if a goroutine holds a mutex indefinitely?
27. Can a `sync.Mutex` be used inside a function?
28. How do you use a mutex in an HTTP server handler?
29. What happens if you call `mu.Lock()` twice without unlocking in between?
30. How do you ensure that a mutex is always unlocked?
31. How do you implement a counter with `sync.Mutex`?
32. Can a mutex protect multiple variables?
33. How do you profile performance bottlenecks related to mutex usage?
34. How do you prevent a goroutine from holding a mutex for too long?
35. What is the impact of excessive mutex usage on application performance?
36. Can `sync.Mutex` be used to synchronize file operations?
37. How do you share a mutex between multiple functions?

- 38.How do you combine `sync.Mutex` with `sync.WaitGroup`?
- 39.How do you ensure fairness when multiple goroutines are waiting for a mutex?
- 40.How do you implement a shared counter with `sync.Mutex`?

## **41-60: Mutex and Data Races**

- 41.What is a data race?
- 42.How do mutexes help prevent data races?
- 43.How do you detect data races in Go?
- 44.What is the `-race` flag in Go?
- 45.How do you resolve a data race using a mutex?
- 46.Can you use `sync.Mutex` to protect a struct field?
- 47.Why should you avoid unlocking a mutex inside a `go` statement?
- 48.Can a mutex prevent read-write conflicts?
- 49.What is the difference between a read lock and a write lock?
- 50.Why is `sync.RWMutex` more efficient for read-heavy operations?
- 51.What happens if two goroutines try to lock the same mutex simultaneously?
- 52.How does Go handle mutex priority when multiple goroutines are waiting?
- 53.What are alternative ways to prevent data races in Go?
- 54.How do mutexes compare to atomic operations?
- 55.Why should you minimize the time a mutex is locked?
- 56.Can a mutex protect non-primitive data types?
- 57.How does a mutex compare to a read-write lock in a database?
- 58.Can a mutex be used for inter-process synchronization?
- 59.What is the difference between `sync.Mutex` and `sync.Cond`?
- 60.How do you reduce contention when multiple goroutines need to acquire a mutex?

## **61-80: Mutex Performance and Best Practices**

- 61.How does `sync.Mutex` affect performance in high-concurrency programs?
- 62.When should you use `sync.RWMutex` instead of `sync.Mutex`?
- 63.How do you reduce mutex contention?
- 64.How do you avoid excessive blocking with mutexes?
- 65.Why should you not hold a mutex while performing I/O operations?
- 66.What are some alternatives to using a mutex?
- 67.How does garbage collection interact with mutexes?
- 68.How do you prevent priority inversion with mutexes?
- 69.How do you implement an efficient queue using `sync.Mutex`?
- 70.Can `sync.Mutex` be used in a distributed system?
- 71.How does Go's runtime handle a mutex when a goroutine is waiting for it?
- 72.Can a mutex be used to synchronize logs?
- 73.How do you design a system that minimizes the need for mutexes?
- 74.Can a mutex be unlocked by a different goroutine?
- 75.What is the difference between `sync.Mutex` and `sync.Once`?
- 76.How does `sync.Mutex` compare to message-passing concurrency?
- 77.What happens if a goroutine that holds a mutex crashes?

- 78.Can you measure mutex contention in Go?
  - 79.What is the impact of using mutexes in a multi-core system?
  - 80.How do you prevent starvation when using a mutex?
- 

## **Advanced Questions (101-150) - Mutex in Go**

### **101-120: Complex Mutex Implementations**

- 101.How do you implement a concurrent linked list using `sync.Mutex`?
- 102.How do you implement a thread-safe cache using `sync.Mutex`?
- 103.How do you implement a rate limiter using `sync.Mutex`?
- 104.How do you implement a priority queue with `sync.Mutex`?
- 105.How do you ensure lock ordering to prevent deadlocks?
- 106.How do you implement a job queue using `sync.Mutex`?
- 107.How do you optimize mutex locks for high-performance applications?
- 108.How do you implement backoff retry logic with `sync.Mutex`?
- 109.How do you measure the impact of mutex contention on CPU usage?
- 110.How do you create a timeout mechanism for mutex acquisition?
- 111.How do you implement a thread-safe registry using `sync.Mutex`?
- 112.How do you use `sync.Mutex` with an in-memory database?
- 113.How do you avoid memory leaks when using mutexes?
- 114.How do you design a system that minimizes mutex dependency?
- 115.How do you combine `sync.Mutex` with atomic counters?
- 116.How do you prevent deadlocks when using multiple mutexes?
- 117.How do you handle contention in a high-performance web server?
- 118.How do you implement a rolling log buffer with `sync.Mutex`?
- 119.How do you efficiently store and retrieve large datasets with mutexes?
- 120.How do you integrate `sync.Mutex` with `context.Context`?

## **Basic Questions (1-100) - Worker Pool in Go**

### **1-20: Worker Pool Basics**

- 1. What is a worker pool in Golang?
- 2. Why is a worker pool useful in concurrent programming?
- 3. How do you implement a basic worker pool in Go?
- 4. What are the main components of a worker pool?
- 5. How do you define the number of workers in a worker pool?
- 6. What is the role of channels in a worker pool?
- 7. How does a worker pool improve performance?
- 8. How do you create worker goroutines in a worker pool?
- 9. What is the difference between a worker pool and a goroutine pool?

- 10.Can a worker pool process multiple tasks concurrently?
- 11.How do you send tasks to workers in a worker pool?
- 12.What happens if the number of workers is too low?
- 13.What happens if the number of workers is too high?
- 14.How do you ensure that all workers finish before the program exits?
- 15.How do you use a `sync.WaitGroup` in a worker pool?
- 16.Can a worker pool handle multiple types of tasks?
- 17.What happens if a worker panics?
- 18.How do you handle worker failures in a worker pool?
- 19.How do you implement a timeout for tasks in a worker pool?
- 20.How do you gracefully shut down a worker pool?

## **21-40: Implementing a Worker Pool**

- 21.How do you structure a worker function?
- 22.How do you limit the number of tasks in a worker pool?
- 23.What data structure is typically used to manage worker tasks?
- 24.How do you prevent memory leaks in a worker pool?
- 25.Can you reuse worker goroutines?
- 26.How do you stop all workers once a certain condition is met?
- 27.How do you implement a worker pool with dynamic scaling?
- 28.How do you ensure that a worker doesn't handle multiple tasks at the same time?
- 29.How do you log worker progress?
- 30.How do you implement worker retries in case of failures?
- 31.How do you measure the performance of a worker pool?
- 32.Can a worker pool be used for database queries?
- 33.How do you avoid race conditions in a worker pool?
- 34.How do you synchronize shared data in a worker pool?
- 35.How do you implement an error handling mechanism in a worker pool?
- 36.What happens if a worker goroutine takes too long to complete?
- 37.How do you monitor the number of active workers?
- 38.How do you handle worker starvation?
- 39.How do you optimize a worker pool for CPU-bound tasks?
- 40.How do you optimize a worker pool for I/O-bound tasks?

## **41-60: Worker Pool with Context and Cancellation**

- 41.How do you implement a worker pool with `context.Context`?
- 42.How do you cancel a task in a worker pool?
- 43.How do you propagate cancellation signals to workers?
- 44.What is the benefit of using `context.WithTimeout` in a worker pool?
- 45.How do you gracefully exit a worker pool when the parent context is canceled?
- 46.Can a worker pool handle task priorities?
- 47.How do you implement a rate-limited worker pool?
- 48.How do you prevent a worker pool from consuming too many system resources?
- 49.How do you debug a worker pool?
- 50.How do you profile memory usage in a worker pool?



- 51.Can workers in a worker pool communicate with each other?
- 52.How do you collect results from a worker pool?
- 53.How do you use `select` with a worker pool?
- 54.How do you implement a worker pool with a buffered channel?
- 55.What are some best practices when designing a worker pool?
- 56.How do you avoid deadlocks in a worker pool?
- 57.How do you ensure fairness in task distribution among workers?
- 58.How do you benchmark a worker pool implementation?
- 59.How do you handle panics inside worker goroutines?
- 60.How do you implement logging in a worker pool?

## **61-80: Worker Pool in Real-World Applications**

- 61.Can a worker pool be used in an HTTP server?
  - 62.How do you use a worker pool for background processing?
  - 63.Can a worker pool be used in batch processing systems?
  - 64.How do you implement a worker pool for processing files concurrently?
  - 65.How do you manage worker pool configuration dynamically?
  - 66.Can a worker pool be used with a message queue?
  - 67.How do you ensure efficient load balancing in a worker pool?
  - 68.How do you test a worker pool in Go?
  - 69.How do you implement a worker pool for distributed systems?
  - 70.How do you prevent goroutine leaks in a worker pool?
  - 71.What are the advantages of using a worker pool over spawning unlimited goroutines?
  - 72.How do you measure task execution time in a worker pool?
  - 73.How do you optimize worker allocation for a CPU-intensive task?
  - 74.How do you implement a worker pool for a web scraping application?
  - 75.How do you implement a worker pool for database migrations?
  - 76.How do you implement a worker pool for a real-time processing system?
  - 77.How do you design a worker pool for high availability?
  - 78.How do you handle logging in a worker pool-based application?
  - 79.Can a worker pool process tasks in batches?
  - 80.How do you combine worker pools with pub-sub systems?
- 

## **Advanced Questions (101-150) - Worker Pool in Go**

### **101-120: Complex Worker Pool Implementations**

- 101.How do you implement a worker pool with a priority queue?
- 102.How do you implement a worker pool with dynamic worker scaling?
- 103.How do you implement a worker pool with task dependencies?
- 104.How do you implement a distributed worker pool across multiple machines?
- 105.How do you handle task retries efficiently in a worker pool?
- 106.How do you track worker status and progress dynamically?
- 107.How do you implement a worker pool with job queues?

- 108.How do you use worker pools with Kubernetes?
- 109.How do you design a worker pool for a microservices architecture?
- 110.How do you implement backpressure in a worker pool?
- 111.How do you implement a worker pool that supports timeouts per task?
- 112.How do you monitor worker pool performance in production?
- 113.How do you handle worker pool failures at scale?
- 114.How do you integrate a worker pool with Redis?
- 115.How do you integrate a worker pool with a message broker like Kafka?
- 116.How do you implement a worker pool for a real-time event processing system?
- 117.How do you use worker pools with AWS Lambda or serverless functions?
- 118.How do you design a worker pool that can process millions of tasks?
- 119.How do you implement a worker pool with a circuit breaker pattern?
- 120.How do you optimize worker pool performance in high-throughput applications?

## **Basic Questions (1-100) - Switch in Go**

### **1-20: Basics of Switch in Go**

1. What is a `switch` statement in Golang?
2. How does `switch` work compared to `if-else` statements?
3. Can `switch` be used without an expression in Golang?
4. What are the benefits of using `switch` instead of multiple `if` conditions?
5. What types of values can be used in a `switch` expression?
6. Can `switch` work with strings in Golang?
7. How does `switch` handle integer values?
8. Can `switch` be used with floating-point numbers?
9. How does `switch` behave with boolean values?
- 10.What happens if no `case` in `switch` matches the value?
- 11.How do you provide a default case in a `switch` statement?
- 12.Can multiple `case` labels have the same body in `switch`?
- 13.Does Golang's `switch` require `break` statements like in C or Java?
- 14.What is an implicit `break` in Golang's `switch`?
- 15.How do you execute multiple cases using `fallthrough`?
- 16.What happens if you use `fallthrough` on the last case?
- 17.Can `switch` work without a condition?
- 18.How does an empty `switch` statement behave?
- 19.What happens if two cases in a `switch` have the same value?
- 20.Can `switch` handle structs as case values?

### **21-40: Switch with Different Data Types**

- 21.Can `switch` be used with `byte` or `rune` values?
- 22.How does `switch` handle `nil` values?

- 23.Can `switch` be used with slices or arrays?
- 24.Why can't you use `switch` with maps in Golang?
- 25.Can `switch` work with `interface{}` types?
- 26.How do you use `switch` to check for `nil` in interfaces?
- 27.How does `switch` work with `const` values?
- 28.Can `switch` handle pointer values?
- 29.Can `switch` work with user-defined types?
- 30.How do you use `switch` with boolean expressions?
- 31.What happens if `switch` is used with multiple expressions in a case?
- 32.Can `switch` be used inside a function?
- 33.Can `switch` be used inside a loop?
- 34.Can `switch` be used to implement simple state machines?
- 35.How do you check multiple values in a single `case` statement?
- 36.How does `switch` handle comparison operations?
- 37.Can you use relational operators (`>`, `<`) in `case` conditions?
- 38.How do you create a `switch` with a range of values?
- 39.Can `switch` work with dynamic values from user input?
- 40.How does `switch` handle `time` types?

## **41-60: Control Flow in Switch**

- 41.Can `switch` be nested inside another `switch`?
- 42.What are the performance implications of `switch` vs. `if-else`?
- 43.How does `switch` handle cases with overlapping values?
- 44.How does `switch` handle empty cases?
- 45.Can `switch` be used for error handling?
- 46.How do you use `switch` with enums in Go?
- 47.Can `switch` be used to match substrings in a string?
- 48.How do you use `switch` to categorize numbers?
- 49.Can `switch` handle negative numbers?
- 50.How do you use `switch` with multiple conditions per case?
- 51.Can `switch` be used to handle HTTP status codes?
- 52.How does `switch` handle case-sensitivity in string matching?
- 53.Can `switch` be used with type assertions?
- 54.How does `switch` interact with closures?
- 55.How do you handle unexpected values in a `switch` statement?
- 56.Can `switch` be used inside a Goroutine?
- 57.How does `switch` interact with context cancellation?
- 58.How does `switch` behave when handling Unicode characters?
- 59.How do you implement a `switch` case for function calls?
- 60.What is the behavior of an empty `default` case in `switch`?

## 61-80: Practical Use Cases of Switch

- 61.How do you use `switch` for string manipulation?
  - 62.Can `switch` be used for parsing command-line arguments?
  - 63.How do you implement a simple menu system using `switch`?
  - 64.Can `switch` be used in unit testing?
  - 65.How does `switch` help in handling API responses?
  - 66.How do you use `switch` in error classification?
  - 67.Can `switch` be used with reflection in Go?
  - 68.How do you handle JSON parsing errors using `switch`?
  - 69.How can `switch` help in implementing role-based access control?
  - 70.How do you optimize `switch` for large numbers of cases?
  - 71.How do you use `switch` to validate user inputs?
  - 72.How does `switch` help in implementing logging levels?
  - 73.How do you use `switch` for handling different database drivers?
  - 74.How does `switch` simplify event-driven programming?
  - 75.Can `switch` be used to build a lightweight router in a web server?
  - 76.How do you implement state transitions using `switch`?
  - 77.How do you use `switch` for handling CLI commands?
  - 78.How do you implement a routing system using `switch` in Go?
  - 79.How does `switch` help in struct field validation?
  - 80.How do you structure `switch` for maintainability in large applications?
- 

## Advanced Questions (101-150) - Switch in Go

### 101-120: Complex Uses of Switch

- 101.How do you implement a `switch` with dynamic case values?
- 102.How do you use `switch` with interfaces dynamically?
- 103.How does `switch` improve code readability in complex conditionals?
- 104.How do you use `switch` with channels in Go?
- 105.How does `switch` interact with reflection and `TypeOf`?
- 106.How can `switch` be used to simplify complex `if-else` chains?
- 107.How do you use `switch` for processing concurrent events?
- 108.How does `switch` interact with functional programming patterns?
- 109.How do you handle unknown types using `switch`?
- 110.How do you implement a finite state machine using `switch`?
- 111.How can `switch` be used with middleware in web applications?
- 112.How do you create a factory function using `switch`?
- 113.How do you prevent fallthrough behavior in complex `switch` logic?
- 114.How do you refactor deeply nested `if-else` into `switch`?
- 115.How do you implement a simple parser using `switch`?
- 116.How do you debug a `switch` statement with many cases?

- 117.How does `switch` handle function return values?
- 118.How do you implement logging strategies using `switch`?
- 119.How do you optimize `switch` statements for performance?
- 120.How do you use `switch` with Goroutines efficiently?

## Basic Questions (1-100) - Select with Loop in Go

### 1-20: Basics of Select Statement

- 1. What is the `select` statement in Golang?
- 2. How is `select` different from a `switch` statement?
- 3. What is the primary use case of the `select` statement?
- 4. How does `select` work with channels?
- 5. Can `select` be used with multiple channels?
- 6. What happens when multiple channels in a `select` statement are ready?
- 7. What happens if no channel is ready in a `select` statement?
- 8. How does `select` handle blocking channels?
- 9. What is a `default` case in a `select` statement?
- 10.How does `select` prevent blocking using `default`?
- 11.What happens if `select` does not have a `default` case and all channels are blocked?
- 12.Can `select` work without any channels?
- 13.What happens if all cases in `select` are unbuffered channels with no data?
- 14.How can `select` be used to implement timeout mechanisms?
- 15.How do you handle multiple channel inputs with `select`?
- 16.Can `select` be used inside a function?
- 17.Can `select` be used inside a Goroutine?
- 18.Is it possible to use `select` inside an `if` statement?
- 19.Can `select` work with a `nil` channel?
- 20.How do you prevent deadlocks using `select`?

### 21-40: Select with Loop (Basic Concepts)

- 21.What happens when `select` is placed inside a `for` loop?
- 22.Can `select` be used inside an infinite loop?
- 23.How does `select` interact with `for` loops?
- 24.What is the purpose of using `select` inside a loop?
- 25.How do you continuously listen to multiple channels using `select`?
- 26.How do you break out of a loop inside a `select`?
- 27.Can `select` be used with `range` loops?
- 28.How does `select` behave when a channel is closed inside a loop?
- 29.What happens if a loop contains a `select` statement with multiple channels?
- 30.How do you ensure `select` inside a loop does not cause high CPU usage?

- 31.Can `select` be used inside a loop with a `time.Sleep`?
- 32.What is the impact of placing a `time.After` inside a loop with `select`?
- 33.How do you exit a `select` loop when a certain condition is met?
- 34.What is an idiomatic way to stop a `select` loop gracefully?
- 35.How does `select` work with multiple Goroutines inside a loop?
- 36.How do you optimize performance when using `select` inside a loop?
- 37.How does `select` handle multiple `default` cases inside a loop?
- 38.How can you use `select` to handle multiple events in a loop?
- 39.What are the risks of using `select` inside an infinite loop?
- 40.How does `select` behave with buffered vs. unbuffered channels inside loops?

## **41-60: Practical Uses of Select with Loop**

- 41.How do you use `select` to process multiple inputs in a loop?
- 42.How can `select` be used for worker pools inside a loop?
- 43.How do you use `select` inside a loop to handle concurrent tasks?
- 44.How do you use `select` to monitor multiple timers?
- 45.Can `select` be used to manage multiple Goroutines inside a loop?
- 46.How does `select` help in implementing a simple event loop?
- 47.How do you use `select` inside a loop to listen for multiple signals?
- 48.How does `select` help in handling server requests inside a loop?
- 49.How can `select` be used to dynamically add new channels inside a loop?
- 50.How do you use `select` with a timeout inside a loop?
- 51.How do you combine `select` and `time.Ticker` in a loop?
- 52.How does `select` inside a loop help in message processing systems?
- 53.How can `select` be used to manage database connections inside a loop?
- 54.How do you handle shutdown signals inside a `select` loop?
- 55.Can `select` be used to handle backpressure inside a loop?
- 56.How can `select` help in preventing memory leaks in loops?
- 57.How does `select` interact with sync primitives inside a loop?
- 58.Can `select` be used to balance workload distribution inside a loop?
- 59.How does `select` inside a loop interact with OS signals?
- 60.How does `select` help in rate-limiting mechanisms?

## **61-80: Handling Edge Cases with Select and Loop**

- 61.What happens if a `select` loop runs indefinitely without yielding?
- 62.How can `select` inside a loop cause Goroutine leaks?
- 63.How do you handle errors when using `select` inside a loop?
- 64.What happens if a channel inside `select` is closed during iteration?
- 65.How do you recover from panics inside a `select` loop?
- 66.What is the behavior of `select` when handling multiple closed channels in a loop?
- 67.How do you restart a `select` loop after an error?
- 68.How do you ensure fairness when `select` inside a loop reads from multiple channels?

- 69.How can `select` be used with logging inside a loop?
  - 70.How do you handle random channel selection inside a `select` loop?
  - 71.What are the performance considerations of using `select` in high-frequency loops?
  - 72.How do you measure execution time of `select` inside a loop?
  - 73.How does garbage collection affect `select` inside a loop?
  - 74.How do you dynamically add new channels inside a `select` loop?
  - 75.How do you handle slow consumers in a `select` loop?
  - 76.What happens if a `select` loop contains blocking operations?
  - 77.How do you prevent starvation in `select` loops?
  - 78.How does `select` behave in nested loops?
  - 79.What debugging techniques help analyze `select` in loops?
  - 80.Can `select` be used inside `defer` statements in a loop?
- 

## Advanced Questions (101-150) - Select with Loop in Go

### 101-120: Performance and Optimization

- 101.How does `select` inside a loop impact memory usage?
- 102.How do you optimize `select` for high-throughput applications?
- 103.What is the role of `runtime.Gosched()` inside a `select` loop?
- 104.How does Go's scheduler affect `select` inside loops?
- 105.How can you use `select` inside a loop to implement efficient event dispatchers?
- 106.How do you tune buffer sizes for channels in `select` loops?
- 107.How do you use `sync.Pool` to optimize `select` inside a loop?
- 108.What are the best practices for writing high-performance `select` loops?
- 109.How do you minimize context-switch overhead in `select` loops?
- 110.What are the implications of using `select` with large numbers of channels?

### 121-150: Complex Use Cases

- 121.How do you implement a priority queue using `select` and loop?
- 122.How do you implement a connection pool using `select` inside a loop?
- 123.How does `select` help in distributed systems inside loops?
- 124.How can `select` inside a loop help in real-time analytics?
- 125.How do you combine `select` with mutexes inside loops?
- 126.Can `select` be used to detect deadlocks inside loops?
- 127.How do you use `select` to build a load balancer?
- 128.How do you handle multi-step workflows using `select` in loops?
- 129.How do you use `select` to implement failover strategies?
- 130.What are alternative approaches to `select` in concurrent loops?

## 1. Context Package in Go

1. What is the purpose of the `context` package in Go?
  2. How do you create a new `context.Context` with a timeout?
  3. What is the difference between `context.WithCancel` and `context.WithTimeout`?
  4. How can you pass a context to a Goroutine and handle cancellation?
  5. What is the use of `context.WithDeadline`, and how is it different from `WithTimeout`?
  6. How does `context.WithValue` work, and what are its use cases?
  7. How do you detect if a context is canceled?
  8. What happens if you don't cancel a context that you create?
  9. Can a context be reused across multiple function calls?
  10. How does `context.TODO()` differ from `context.Background()`?
- 

## 2. Memory Management in Go

1. How does Go manage memory allocation and deallocation?
  2. What is the difference between stack and heap allocation in Go?
  3. How does Go's garbage collector work?
  4. What are the advantages and disadvantages of automatic memory management in Go?
  5. How can you manually reduce memory usage in Go applications?
  6. What tools does Go provide for memory profiling and debugging?
  7. What is the impact of large object allocations on Go's performance?
  8. How do escape analysis and heap allocations affect performance?
  9. What are memory leaks, and how can you detect them in Go?
  10. How can `sync.Pool` be used to optimize memory usage?
- 

## 3. Building a Custom Scheduler in Go

1. How does Go's default scheduler handle Goroutines?
  2. What are the key components of Go's runtime scheduler?
  3. How can you create a custom scheduler for specific use cases?
  4. How does Go balance Goroutines across multiple OS threads?
  5. What are G, M, and P in Go's runtime?
  6. How does work-stealing work in Go's scheduler?
  7. How does Go handle preemption of long-running Goroutines?
  8. What are the limitations of modifying Go's scheduler?
  9. How does `GOMAXPROCS` affect scheduling decisions?
  10. How can you debug and profile Go's scheduler behavior?
-



## 4. Atomic Operations and Sync/Atomic

1. What is the purpose of the `sync/atomic` package?
  2. How does `atomic.AddInt64` work, and why is it useful?
  3. What is a compare-and-swap (CAS) operation in Go?
  4. When should you use atomic operations instead of mutexes?
  5. What are the limitations of atomic operations?
  6. How can you implement a lock-free data structure using atomic operations?
  7. How does `atomic.Value` help in safe concurrent access?
  8. What is the purpose of `atomic.Load` and `atomic.Store`?
  9. Can atomic operations cause performance bottlenecks?
  10. How does memory alignment affect atomic operations?
- 

## 5. Profiling and Benchmarking in Go

1. How do you use `pprof` for CPU profiling in Go?
  2. What is heap profiling, and how does Go handle it?
  3. How do you benchmark a function using the `testing` package?
  4. What is the difference between `b.N` and `b.ResetTimer()` in benchmarks?
  5. How can you analyze Goroutine leaks using profiling tools?
  6. How does `trace` differ from `pprof` in Go?
  7. What are flame graphs, and how can they be used to optimize performance?
  8. How can you measure memory allocations per operation in benchmarks?
  9. How do you visualize profiling data using `go tool pprof`?
  10. What are some best practices for benchmarking in Go?
- 

## 6. Implementing Plugins with Go

1. What is the `plugin` package in Go used for?
  2. How do you load a shared object file (`.so`) in Go?
  3. What are the limitations of Go's plugin system?
  4. How do you define and expose functions in a Go plugin?
  5. What is `plugin.Lookup()`, and how does it work?
  6. How do you handle versioning when using Go plugins?
  7. Can plugins be dynamically loaded and unloaded at runtime?
  8. How does Go's static linking affect plugins?
  9. What are alternative approaches to plugins in Go?
  10. How can plugins be used for extending a Go application?
- 

## 7. Building a Web Framework from Scratch

1. What are the core components of a web framework?

2. How do you implement an HTTP router in Go?
  3. What is middleware, and how do you implement it in a Go web framework?
  4. How do you handle request and response parsing in Go?
  5. What are the benefits of using `net/http` as the base for a web framework?
  6. How do you implement dependency injection in a Go web framework?
  7. How does Go handle concurrency in web servers?
  8. How do you implement session management in a custom framework?
  9. How do you add support for JSON and XML in a web framework?
  10. What are the key performance considerations when building a web framework?
- 

## 8. Understanding Go's Runtime Internals

1. How does Go's runtime manage Goroutines?
  2. What is the purpose of Go's garbage collector?
  3. How does Go's scheduler distribute Goroutines across threads?
  4. What is escape analysis, and how does it affect memory allocation?
  5. How does Go's stack management work?
  6. What is `sysmon`, and how does it impact performance?
  7. How does Go handle preemption of long-running Goroutines?
  8. What are the key runtime optimizations in Go?
  9. How does Go's runtime handle system calls?
  10. How can you modify Go's runtime behavior for performance tuning?
- 

## 9. Using Reflection in Go

1. What is reflection in Go, and when is it useful?
  2. How do you inspect the type of a variable using reflection?
  3. How do you modify the value of a struct field using reflection?
  4. What is `reflect.TypeOf()` and `reflect.ValueOf()`?
  5. How do you check if an interface implements a specific type using reflection?
  6. How does reflection impact performance in Go?
  7. How can you use reflection to implement a generic function?
  8. What are common pitfalls when using reflection?
  9. How do you dynamically invoke methods using reflection?
  10. When should you avoid using reflection in Go?
- 

## 10. Writing a Custom DSL in Go

1. What is a Domain-Specific Language (DSL)?
2. What are the benefits of implementing a DSL in Go?
3. How do you tokenize a custom language in Go?
4. What is an Abstract Syntax Tree (AST), and how do you build one in Go?

5. How do you implement a parser for a DSL in Go?
6. What are some common use cases for a DSL?
7. How do you execute a custom DSL in Go?
8. How does Go's `text/template` and `go/parser` packages help in DSL creation?
9. How do you handle errors in a DSL interpreter?
10. What are some examples of DSLs written in Go?